8.1 General Trees

Productivity experts say that breakthroughs come by thinking "nonlinearly." In this chapter, we discuss one of the most important nonlinear data structures in computing—***trees***. Tree structures are indeed a breakthrough in data organization, for they allow us to implement a host of algorithms much faster than when using linear data structures, such as array-based lists or linked lists. Trees also provide a natural organization for data, and consequently have become ubiquitous structures in file systems, graphical user interfaces, databases, Web sites, and other computer systems.

It is not always clear what productivity experts mean by "nonlinear" thinking, but when we say that trees are "nonlinear," we are referring to an organizational relationship that is richer than the simple "before" and "after" relationships between objects in sequences. The relationships in a tree are ***hierarchical***, with some objects being "above" and some "below" others. Actually, the main terminology for tree data structures comes from family trees, with the terms "parent," "child," "ancestor," and "descendant" being the most common words used to describe relationships. We show an example of a family tree in Figure 8.1.

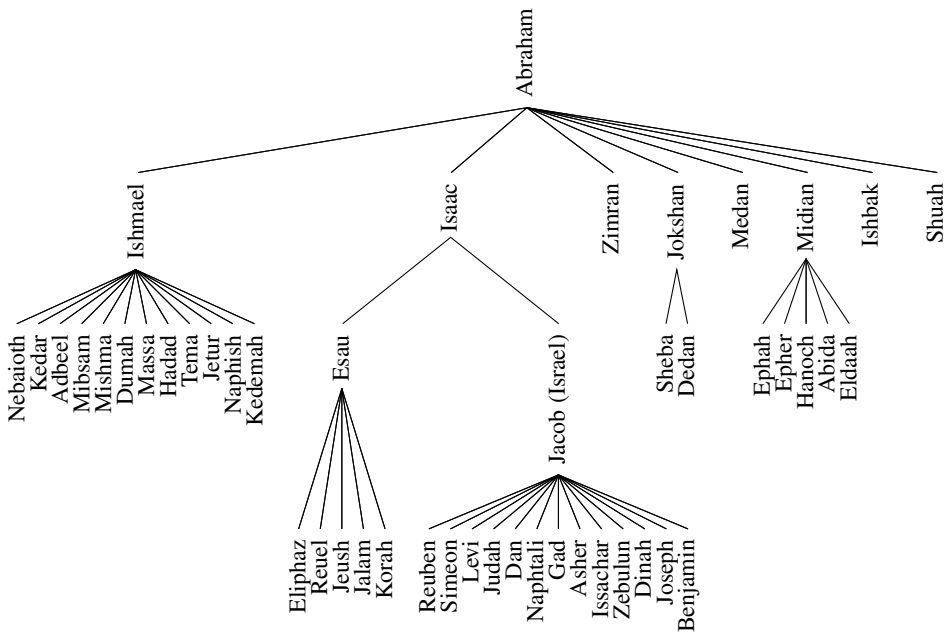


Figure 8.1: A family tree showing some descendants of Abraham, as recorded in Genesis, chapters 25–36.

8.1.1 Tree Definitions and Properties

A ***tree*** is an abstract data type that stores elements hierarchically. With the exception of the top element, each element in a tree has a ***parent*** element and zero or more ***children*** elements. A tree is usually visualized by placing elements inside ovals or rectangles, and by drawing the connections between parents and children with straight lines. (See Figure 8.2.) We typically call the top element the ***root*** of the tree, but it is drawn as the highest element, with the other elements being connected below (just the opposite of a botanical tree).

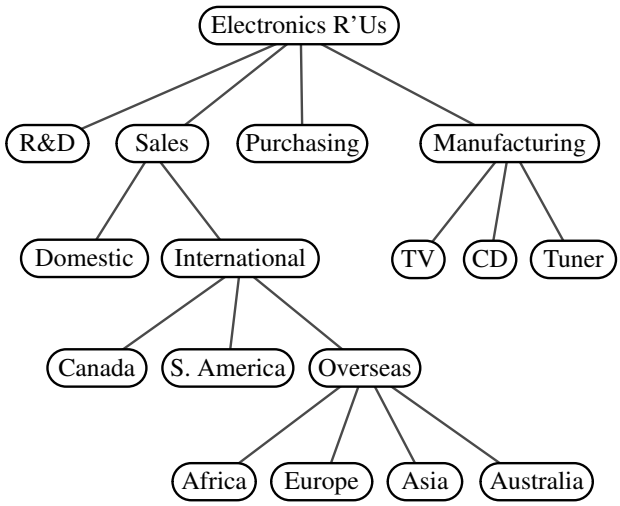


Figure 8.2: A tree with 17 nodes representing the organization of a fictitious corporation. The root stores *Electronics R'Us*. The children of the root store *R&D*, *Sales*, *Purchasing*, and *Manufacturing*. The internal nodes store *Sales*, *International*, *Overseas*, *Electronics R'Us*, and *Manufacturing*.

Formal Tree Definition

Formally, we define a ***tree*** T as a set of ***nodes*** storing elements such that the nodes have a ***parent-child*** relationship that satisfies the following properties:

- If T is nonempty, it has a special node, called the ***root*** of T, that has no parent.
- Each node v of T different from the root has a unique ***parent*** node w; every node with parent w is a ***child*** of w.

Note that according to our definition, a tree can be empty, meaning that it does not have any nodes. This convention also allows us to define a tree recursively such that a tree T is either empty or consists of a node r, called the root of T, and a (possibly empty) set of subtrees whose roots are the children of r.

Other Node Relationships

Two nodes that are children of the same parent are ***siblings***. A node v is ***external*** if v has no children. A node v is ***internal*** if it has one or more children. External nodes are also known as ***leaves***.

Example 8.1: *In Section 4.1.4, we discussed the hierarchical relationship between files and directories in a computer's file system, although at the time we did not emphasize the nomenclature of a file system as a tree. In Figure 8.3, we revisit an earlier example. We see that the internal nodes of the tree are associated with directories and the leaves are associated with regular files. In the UNIX and Linux operating systems, the root of the tree is appropriately called the "root directory," and is represented by the symbol "/."*

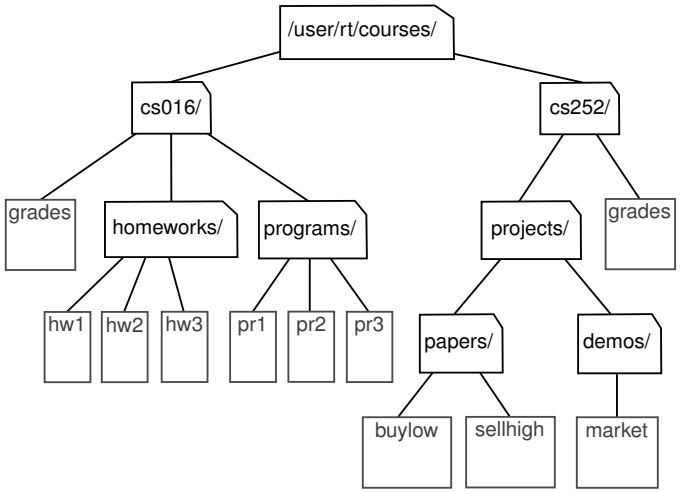


Figure 8.3: Tree representing a portion of a file system.

A node u is an ***ancestor*** of a node v if $u = v$ or u is an ancestor of the parent of v. Conversely, we say that a node v is a ***descendant*** of a node u if u is an ancestor of v. For example, in Figure 8.3, cs252/ is an ancestor of papers/, and pr3 is a descendant of cs016/. The ***subtree*** of T ***rooted*** at a node v is the tree consisting of all the descendants of v in T (including v itself). In Figure 8.3, the subtree rooted at cs016/ consists of the nodes cs016/, grades, homeworks/, programs/, hw1, hw2, hw3, pr1, pr2, and pr3.

Edges and Paths in Trees

An ***edge*** of tree T is a pair of nodes (u,v) such that u is the parent of v, or vice versa. A ***path*** of T is a sequence of nodes such that any two consecutive nodes in the sequence form an edge. For example, the tree in Figure 8.3 contains the path (cs252/, projects/, demos/, market).

Example 8.2: *The inheritance relation between classes in a Python program forms a tree when single inheritance is used. For example, in Section 2.4 we provided a summary of the hierarchy for Python's exception types, as portrayed in Figure 8.4 (originally Figure 2.5). The BaseException class is the root of that hierarchy, while all user-defined exception classes should conventionally be declared as descendants of the more specific Exception class. (See, for example, the Empty class we introduced in Code Fragment 6.1 of Chapter 6.)*

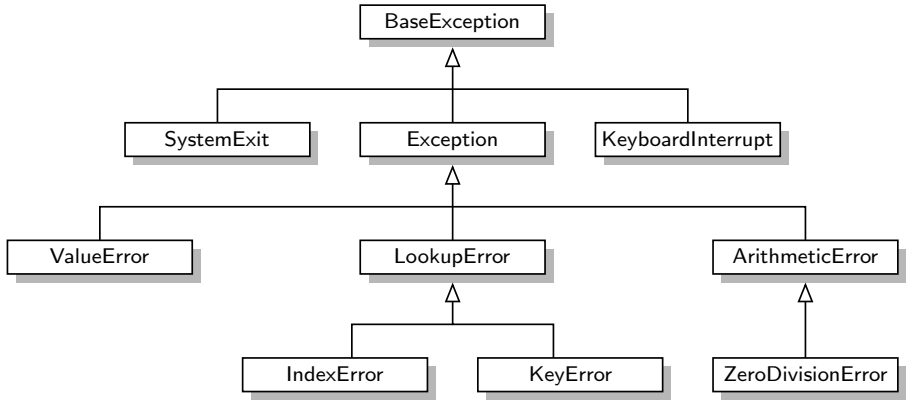


Figure 8.4: A portion of Python's hierarchy of exception types.

In Python, all classes are organized into a single hierarchy, as there exists a built-in class named object as the ultimate base class. It is a direct or indirect base class of all other types in Python (even if not declared as such when defining a new class). Therefore, the hierarchy pictured in Figure 8.4 is only a portion of Python's complete class hierarchy.

As a preview of the remainder of this chapter, Figure 8.5 portrays our own hierarchy of classes for representing various forms of a tree.

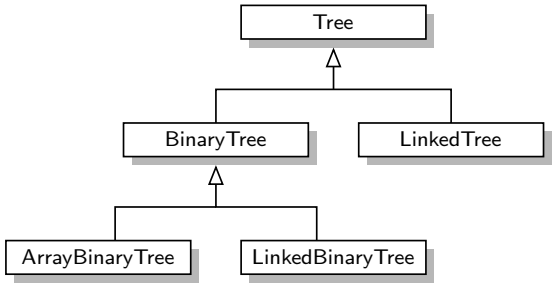


Figure 8.5: Our own inheritance hierarchy for modeling various abstractions and implementations of tree data structures. In the remainder of this chapter, we provide implementations of Tree, BinaryTree, and LinkedBinaryTree classes, and high-level sketches for how LinkedTree and ArrayBinaryTree might be designed.

Ordered Trees

A tree is ***ordered*** if there is a meaningful linear order among the children of each node; that is, we purposefully identify the children of a node as being the first, second, third, and so on. Such an order is usually visualized by arranging siblings left to right, according to their order.

Example 8.3: *The components of a structured document, such as a book, are hierarchically organized as a tree whose internal nodes are parts, chapters, and sections, and whose leaves are paragraphs, tables, figures, and so on. (See Figure 8.6.) The root of the tree corresponds to the book itself. We could, in fact, consider expanding the tree further to show paragraphs consisting of sentences, sentences consisting of words, and words consisting of characters. Such a tree is an example of an ordered tree, because there is a well-defined order among the children of each node.*

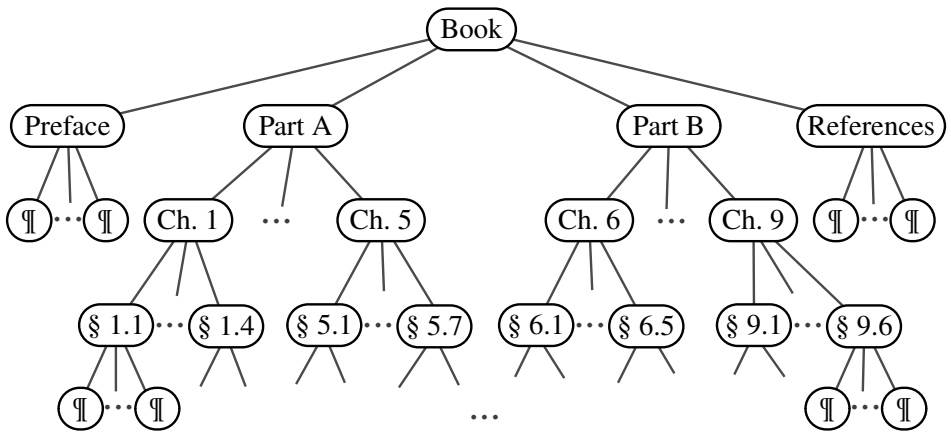


Figure 8.6: An ordered tree associated with a book.

Let's look back at the other examples of trees that we have described thus far, and consider whether the order of children is significant. A family tree that describes generational relationships, as in Figure 8.1, is often modeled as an ordered tree, with siblings ordered according to their birth.

In contrast, an organizational chart for a company, as in Figure 8.2, is typically considered an unordered tree. Likewise, when using a tree to describe an inheritance hierarchy, as in Figure 8.4, there is no particular significance to the order among the subclasses of a parent class. Finally, we consider the use of a tree in modeling a computer's file system, as in Figure 8.3. Although an operating system often displays entries of a directory in a particular order (e.g., alphabetical, chronological), such an order is not typically inherent to the file system's representation.

8.1.2 The Tree Abstract Data Type

As we did with positional lists in Section 7.4, we define a tree ADT using the concept of a ***position*** as an abstraction for a node of a tree. An element is stored at each position, and positions satisfy parent-child relationships that define the tree structure. A position object for a tree supports the method:

p.element( ): Return the element stored at position p.

The tree ADT then supports the following ***accessor methods***, allowing a user to navigate the various positions of a tree:

T.root( ): Return the position of the root of tree T, or None if T is empty.

T.is_root(p): Return True if position p is the root of Tree T.

T.parent(p): Return the position of the parent of position p, or None if p is the root of T.

T.num_children(p): Return the number of children of position p.

T.children(p): Generate an iteration of the children of position p.

T.is_leaf(p): Return True if position p does not have any children.

len(T): Return the number of positions (and hence elements) that are contained in tree T.

T.is_empty( ): Return True if tree T does not contain any positions.

T.positions( ): Generate an iteration of all *positions* of tree T.

iter(T): Generate an iteration of all *elements* stored within tree T.

Any of the above methods that accepts a position as an argument should generate a ValueError if that position is invalid for T.

If a tree T is ordered, then T.children(p) reports the children of p in the natural order. If p is a leaf, then T.children(p) generates an empty iteration. In similar regard, if tree T is empty, then both T.positions( ) and iter(T) generate empty iterations. We will discuss general means for iterating through all positions of a tree in Sections 8.4.

We do not define any methods for creating or modifying trees at this point. We prefer to describe different tree update methods in conjunction with specific implementations of the tree interface, and specific applications of trees.

A Tree Abstract Base Class in Python

In discussing the object-oriented design principle of abstraction in Section 2.1.2, we noted that a public interface for an abstract data type is often managed in Python via ***duck typing***. For example, we defined the notion of the public interface for a queue ADT in Section 6.2, and have since presented several classes that implement the queue interface (e.g., ArrayQueue in Section 6.2.2, LinkedQueue in Section 7.1.2, CircularQueue in Section 7.2.2). However, we never gave any formal definition of the queue ADT in Python; all of the concrete implementations were self-contained classes that just happen to adhere to the same public interface. A more formal mechanism to designate the relationships between different implementations of the same abstraction is through the definition of one class that serves as an ***abstract base class***, via inheritance, for one or more ***concrete classes***. (See Section 2.4.3.)

We choose to define a Tree class, in Code Fragment 8.1, that serves as an abstract base class corresponding to the tree ADT. Our reason for doing so is that there is quite a bit of useful code that we can provide, even at this level of abstraction, allowing greater code reuse in the concrete tree implementations we later define. The Tree class provides a definition of a nested Position class (which is also abstract), and declarations of many of the accessor methods included in the tree ADT.

However, our Tree class does not define any internal representation for storing a tree, and five of the methods given in that code fragment remain ***abstract*** (root, parent, num_children, children, and __len__); each of these methods raises a NotImplementedError. (A more formal approach for defining abstract base classes and abstract methods, using Python's abc module, is described in Section 2.4.3.) The subclasses are responsible for overriding abstract methods, such as children, to provide a working implementation for each behavior, based on their chosen internal representation.

Although the Tree class is an abstract base class, it includes several ***concrete*** methods with implementations that rely on calls to the abstract methods of the class. In defining the tree ADT in the previous section, we declare ten accessor methods. Five of those are the ones we left as abstract, in Code Fragment 8.1. The other five can be implemented based on the former. Code Fragment 8.2 provides concrete implementations for methods is_root, is_leaf, and is_empty. In Section 8.4, we will explore general algorithms for traversing a tree that can be used to provide concrete implementations of the positions and __iter__ methods within the Tree class. The beauty of this design is that the concrete methods defined within the Tree abstract base class will be inherited by all subclasses. This promotes greater code reuse, as there will be no need for those subclasses to reimplement such behaviors.

We note that, with the Tree class being abstract, there is no reason to create a direct instance of it, nor would such an instance be useful. The class exists to serve as a base for inheritance, and users will create instances of concrete subclasses.

```
class Tree:
  """Abstract base class representing a tree structure."""

  #------------------------------ nested Position class ------------------------------
  class Position:
    """An abstraction representing the location of a single element."""

    def element(self):
      """Return the element stored at this Position."""
      raise NotImplementedError('must be implemented by subclass')

    def __eq__(self, other):
      """Return True if other Position represents the same location."""
      raise NotImplementedError('must be implemented by subclass')

    def __ne__(self, other):
      """Return True if other does not represent the same location."""
      return not (self == other)                # opposite of __eq__

  # ---------- abstract methods that concrete subclass must support ----------
  def root(self):
    """Return Position representing the tree's root (or None if empty)."""
    raise NotImplementedError('must be implemented by subclass')

  def parent(self, p):
    """Return Position representing p's parent (or None if p is root)."""
    raise NotImplementedError('must be implemented by subclass')

  def num_children(self, p):
    """Return the number of children that Position p has."""
    raise NotImplementedError('must be implemented by subclass')

  def children(self, p):
    """Generate an iteration of Positions representing p's children."""
    raise NotImplementedError('must be implemented by subclass')

  def __len__(self):
    """Return the total number of elements in the tree."""
    raise NotImplementedError('must be implemented by subclass')
```

Code Fragment 8.1: A portion of our Tree abstract base class (continued in Code Fragment 8.2).

```
40    # ---------- concrete methods implemented in this class ----------
41    def is_root(self, p):
42      """Return True if Position p represents the root of the tree."""
43      return self.root( ) == p
44
45    def is_leaf(self, p):
46      """Return True if Position p does not have any children."""
47      return self.num_children(p) == 0
48
49    def is_empty(self):
50      """Return True if the tree is empty."""
51      return len(self) == 0
```

Code Fragment 8.2: Some concrete methods of our Tree abstract base class.

8.1.3 Computing Depth and Height

Let p be the position of a node of a tree T. The ***depth*** of p is the number of ancestors of p, excluding p itself. For example, in the tree of Figure 8.2, the node storing *International* has depth 2. Note that this definition implies that the depth of the root of T is 0. The depth of p can also be recursively defined as follows:

- If p is the root, then the depth of p is 0.
- Otherwise, the depth of p is one plus the depth of the parent of p.

Based on this definition, we present a simple, recursive algorithm, depth, in Code Fragment 8.3, for computing the depth of a position p in Tree T. This method calls itself recursively on the parent of p, and adds 1 to the value returned.

```
52    def depth(self, p):
53      """Return the number of levels separating Position p from the root."""
54      if self.is_root(p):
55        return 0
56      else:
57        return 1 + self.depth(self.parent(p))
```

Code Fragment 8.3: Method depth of the Tree class.

The running time of T.depth(p) for position p is $O(d_p + 1)$, where d_p denotes the depth of p in the tree T, because the algorithm performs a constant-time recursive step for each ancestor of p. Thus, algorithm T.depth(p) runs in $O(n)$ worst-case time, where n is the total number of positions of T, because a position of T may have depth $n-1$ if all nodes form a single branch. Although such a running time is a function of the input size, it is more informative to characterize the running time in terms of the parameter d_p, as this parameter may be much smaller than n.

Height

The ***height*** of a position p in a tree T is also defined recursively:

- If p is a leaf, then the height of p is 0.
- Otherwise, the height of p is one more than the maximum of the heights of p's children.

The ***height*** of a nonempty tree T is the height of the root of T. For example, the tree of Figure 8.2 has height 4. In addition, height can also be viewed as follows.

Proposition 8.4: *The height of a nonempty tree T is equal to the maximum of the depths of its leaf positions.*

We leave the justification of this fact to an exercise (R-8.3). We present an algorithm, height1, implemented in Code Fragment 8.4 as a nonpublic method _height1 of the Tree class. It computes the height of a nonempty tree T based on Proposition 8.4 and the algorithm depth from Code Fragment 8.3.

```
58   def _height1(self):                    # works, but O(n^2) worst-case time
59     """Return the height of the tree."""
60     return max(self.depth(p) for p in self.positions( ) if self.is_leaf(p))
```

Code Fragment 8.4: Method _height1 of the Tree class. Note that this method calls the depth method.

Unfortunately, algorithm height1 is not very efficient. We have not yet defined the positions( ) method; we will see that it can be implemented to run in $O(n)$ time, where n is the number of positions of T. Because height1 calls algorithm depth(p) on each leaf of T, its running time is $O(n + \sum_{p \in L}(d_p + 1))$, where L is the set of leaf positions of T. In the worst case, the sum $\sum_{p \in L}(d_p + 1)$ is proportional to n^2. (See Exercise C-8.33.) Thus, algorithm height1 runs in $O(n^2)$ worst-case time.

We can compute the height of a tree more efficiently, in $O(n)$ worst-case time, by relying instead on the original recursive definition. To do this, we will parameterize a function based on a position within the tree, and calculate the height of the subtree rooted at that position. Algorithm height2, shown as nonpublic method _height2 in Code Fragment 8.5, computes the height of tree T in this way.

```
61   def _height2(self, p):                 # time is linear in size of subtree
62     """Return the height of the subtree rooted at Position p."""
63     if self.is_leaf(p):
64       return 0
65     else:
66       return 1 + max(self._height2(c) for c in self.children(p))
```

Code Fragment 8.5: Method _height2 for computing the height of a subtree rooted at a position p of a Tree.

It is important to understand why algorithm height2 is more efficient than height1. The algorithm is recursive, and it progresses in a top-down fashion. If the method is initially called on the root of T, it will eventually be called once for each position of T. This is because the root eventually invokes the recursion on each of its children, which in turn invokes the recursion on each of their children, and so on.

We can determine the running time of the height2 algorithm by summing, over all the positions, the amount of time spent on the nonrecursive part of each call. (Review Section 4.2 for analyses of recursive processes.) In our implementation, there is a constant amount of work per position, plus the overhead of computing the maximum over the iteration of children. Although we do not yet have a concrete implementation of children(p), we assume that such an iteration is generated in $O(c_p + 1)$ time, where c_p denotes the number of children of p. Algorithm height2 spends $O(c_p + 1)$ time at each position p to compute the maximum, and its overall running time is $O(\sum_p (c_p + 1)) = O(n + \sum_p c_p)$. In order to complete the analysis, we make use of the following property.

Proposition 8.5: *Let T be a tree with n positions, and let c_p denote the number of children of a position p of T. Then, summing over the positions of T, $\sum_p c_p = n - 1$.*

Justification: Each position of T, with the exception of the root, is a child of another position, and thus contributes one unit to the above sum. ■

By Proposition 8.5, the running time of algorithm height2, when called on the root of T, is $O(n)$, where n is the number of positions of T.

Revisiting the public interface for our Tree class, the ability to compute heights of subtrees is beneficial, but a user might expect to be able to compute the height of the entire tree without explicitly designating the tree root. We can wrap the non-public _height2 in our implementation with a public height method that provides a default interpretation when invoked on tree T with syntax T.height( ). Such an implementation is given in Code Fragment 8.6.

```
67   def height(self, p=None):
68     """Return the height of the subtree rooted at Position p.
69
70     If p is None, return the height of the entire tree.
71     """
72     if p is None:
73       p = self.root( )
74     return self._height2(p)          # start _height2 recursion
```

Code Fragment 8.6: Public method Tree.height that computes the height of the entire tree by default, or a subtree rooted at given position, if specified.

8.2 Binary Trees

A ***binary tree*** is an ordered tree with the following properties:

1. Every node has at most two children.
2. Each child node is labeled as being either a ***left child*** or a ***right child***.
3. A left child precedes a right child in the order of children of a node.

The subtree rooted at a left or right child of an internal node v is called a ***left subtree*** or ***right subtree***, respectively, of v. A binary tree is ***proper*** if each node has either zero or two children. Some people also refer to such trees as being ***full*** binary trees. Thus, in a proper binary tree, every internal node has exactly two children. A binary tree that is not proper is ***improper***.

Example 8.6: *An important class of binary trees arises in contexts where we wish to represent a number of different outcomes that can result from answering a series of yes-or-no questions. Each internal node is associated with a question. Starting at the root, we go to the left or right child of the current node, depending on whether the answer to the question is "Yes" or "No." With each decision, we follow an edge from a parent to a child, eventually tracing a path in the tree from the root to a leaf. Such binary trees are known as* **decision trees**, *because a leaf position p in such a tree represents a decision of what to do if the questions associated with p's ancestors are answered in a way that leads to p. A decision tree is a proper binary tree. Figure 8.7 illustrates a decision tree that provides recommendations to a prospective investor.*

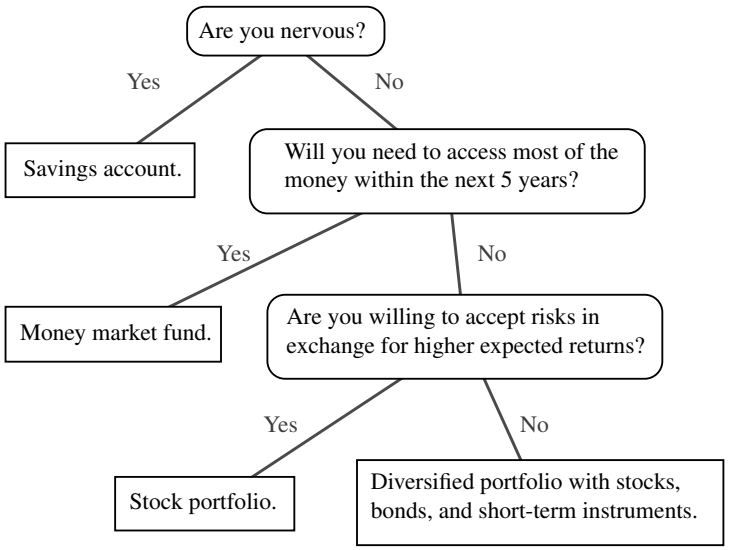


Figure 8.7: A decision tree providing investment advice.

Example 8.7: *An arithmetic expression can be represented by a binary tree whose leaves are associated with variables or constants, and whose internal nodes are associated with one of the operators $+$, $-$, $\times$, and $/$. (See Figure 8.8.) Each node in such a tree has a value associated with it.*

- *If a node is leaf, then its value is that of its variable or constant.*
- *If a node is internal, then its value is defined by applying its operation to the values of its children.*

An arithmetic expression tree is a proper binary tree, since each operator $+$, $-$, $\times$, and $/$ takes exactly two operands. Of course, if we were to allow unary operators, like negation ($-$), as in "$-x$," then we could have an improper binary tree.

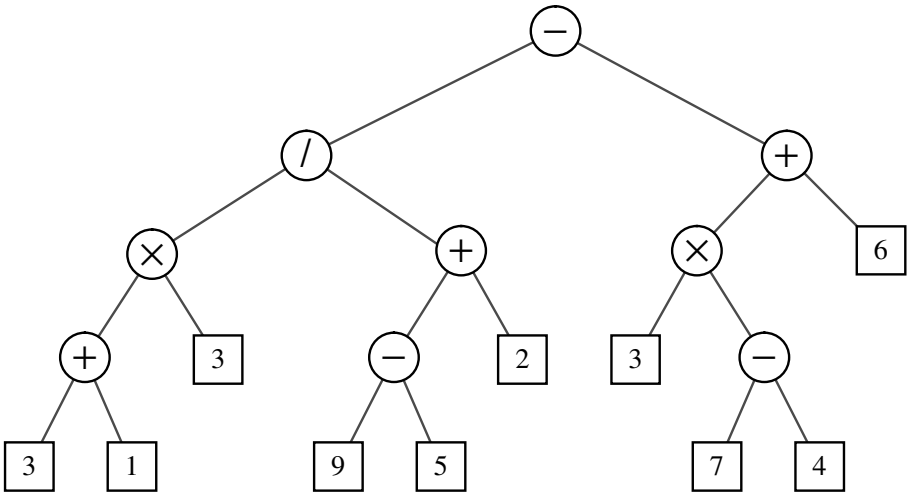


Figure 8.8: A binary tree representing an arithmetic expression. This tree represents the expression $((((3+1)\times 3)/((9-5)+2)) - ((3\times(7-4))+6))$. The value associated with the internal node labeled "/" is 2.

A Recursive Binary Tree Definition

Incidentally, we can also define a binary tree in a recursive way such that a binary tree is either empty or consists of:

- A node r, called the root of T, that stores an element
- A binary tree (possibly empty), called the left subtree of T
- A binary tree (possibly empty), called the right subtree of T

8.2.1 The Binary Tree Abstract Data Type

As an abstract data type, a binary tree is a specialization of a tree that supports three additional accessor methods:

T.left(p): Return the position that represents the left child of p, or None if p has no left child.

T.right(p): Return the position that represents the right child of p, or None if p has no right child.

T.sibling(p): Return the position that represents the sibling of p, or None if p has no sibling.

Just as in Section 8.1.2 for the tree ADT, we do not define specialized update methods for binary trees here. Instead, we will consider some possible update methods when we describe specific implementations and applications of binary trees.

The BinaryTree Abstract Base Class in Python

Just as Tree was defined as an abstract base class in Section 8.1.2, we define a new BinaryTree class associated with the binary tree ADT. We rely on inheritance to define the BinaryTree class based upon the existing Tree class. However, our BinaryTree class remains *abstract*, as we still do not provide complete specifications for how such a structure will be represented internally, nor implementations for some necessary behaviors.

Our Python implementation of the BinaryTree class is given in Code Fragment 8.7. By using inheritance, a binary tree supports all the functionality that was defined for general trees (e.g., parent, is_leaf, root). Our new class also inherits the nested Position class that was originally defined within the Tree class definition. In addition, the new class provides declarations for new abstract methods left and right that should be supported by concrete subclasses of BinaryTree.

Our new class also provides two concrete implementations of methods. The new sibling method is derived from the combination of left, right, and parent. Typically, we identify the sibling of a position p as the "other" child of p's parent. However, if p is the root, it has no parent, and thus no sibling. Also, p may be the only child of its parent, and thus does not have a sibling.

Finally, Code Fragment 8.7 provides a concrete implementation of the children method; this method is abstract in the Tree class. Although we have still not specified how the children of a node will be stored, we derive a generator for the ordered children based upon the implied behavior of abstract methods left and right.

```
class BinaryTree(Tree):
  """Abstract base class representing a binary tree structure."""

  # --------------------- additional abstract methods ---------------------
  def left(self, p):
    """Return a Position representing p's left child.

    Return None if p does not have a left child.
    """
    raise NotImplementedError('must be implemented by subclass')

  def right(self, p):
    """Return a Position representing p's right child.

    Return None if p does not have a right child.
    """
    raise NotImplementedError('must be implemented by subclass')

  # ---------- concrete methods implemented in this class ----------
  def sibling(self, p):
    """Return a Position representing p's sibling (or None if no sibling)."""
    parent = self.parent(p)
    if parent is None:                    # p must be the root
      return None                         # root has no sibling
    else:
      if p == self.left(parent):
        return self.right(parent)         # possibly None
      else:
        return self.left(parent)          # possibly None

  def children(self, p):
    """Generate an iteration of Positions representing p's children."""
    if self.left(p) is not None:
      yield self.left(p)
    if self.right(p) is not None:
      yield self.right(p)
```

Code Fragment 8.7: A BinaryTree abstract base class that extends the existing Tree abstract base class from Code Fragments 8.1 and 8.2.

8.2.2 Properties of Binary Trees

Binary trees have several interesting properties dealing with relationships between their heights and number of nodes. We denote the set of all nodes of a tree T at the same depth d as ***level*** d of T. In a binary tree, level 0 has at most one node (the root), level 1 has at most two nodes (the children of the root), level 2 has at most four nodes, and so on. (See Figure 8.9.) In general, level d has at most 2^d nodes.

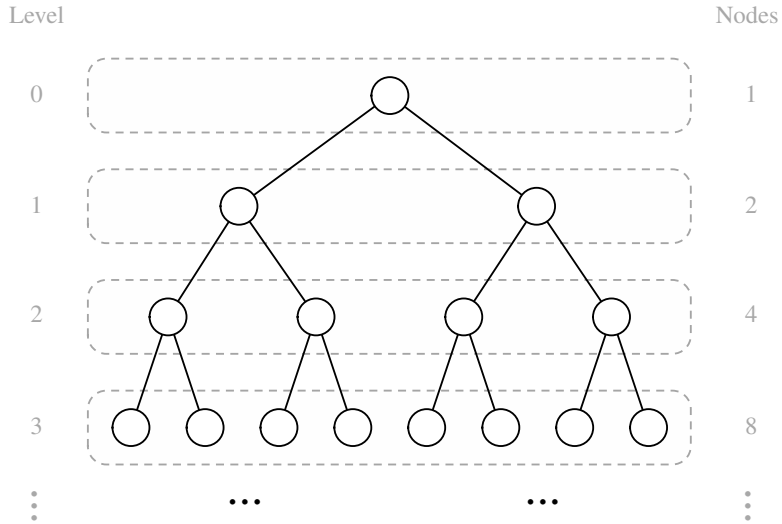


Figure 8.9: Maximum number of nodes in the levels of a binary tree.

We can see that the maximum number of nodes on the levels of a binary tree grows exponentially as we go down the tree. From this simple observation, we can derive the following properties relating the height of a binary tree T with its number of nodes. A detailed justification of these properties is left as Exercise R-8.8.

Proposition 8.8: *Let T be a nonempty binary tree, and let n, n_E, n_I and h denote the number of nodes, number of external nodes, number of internal nodes, and height of T, respectively. Then T has the following properties:*

1. $h+1 \leq n \leq 2^{h+1}-1$
2. $1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h-1$
4. $\log(n+1)-1 \leq h \leq n-1$

Also, if T is proper, then T has the following properties:

1. $2h+1 \leq n \leq 2^{h+1}-1$
2. $h+1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h-1$
4. $\log(n+1)-1 \leq h \leq (n-1)/2$

Relating Internal Nodes to External Nodes in a Proper Binary Tree

In addition to the earlier binary tree properties, the following relationship exists between the number of internal nodes and external nodes in a proper binary tree.

Proposition 8.9: *In a nonempty proper binary tree T, with n_E external nodes and n_I internal nodes, we have $n_E = n_I + 1$.*

Justification: We justify this proposition by removing nodes from T and dividing them up into two "piles," an internal-node pile and an external-node pile, until T becomes empty. The piles are initially empty. By the end, we will show that the external-node pile has one more node than the internal-node pile. We consider two cases:

Case 1: If T has only one node v, we remove v and place it on the external-node pile. Thus, the external-node pile has one node and the internal-node pile is empty.

Case 2: Otherwise (T has more than one node), we remove from T an (arbitrary) external node w and its parent v, which is an internal node. We place w on the external-node pile and v on the internal-node pile. If v has a parent u, then we reconnect u with the former sibling z of w, as shown in Figure 8.10. This operation, removes one internal node and one external node, and leaves the tree being a proper binary tree.

Repeating this operation, we eventually are left with a final tree consisting of a single node. Note that the same number of external and internal nodes have been removed and placed on their respective piles by the sequence of operations leading to this final tree. Now, we remove the node of the final tree and we place it on the external-node pile. Thus, the the external-node pile has one more node than the internal-node pile.

■

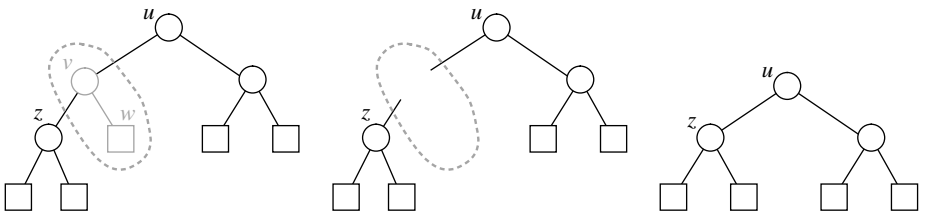


Figure 8.10: Operation that removes an external node and its parent node, used in the justification of Proposition 8.9.

Note that the above relationship does not hold, in general, for improper binary trees and nonbinary trees, although there are other interesting relationships that do hold. (See Exercises C-8.32 through C-8.34.)

8.3 Implementing Trees

The Tree and BinaryTree classes that we have defined thus far in this chapter are both formally ***abstract base classes***. Although they provide a great deal of support, neither of them can be directly instantiated. We have not yet defined key implementation details for how a tree will be represented internally, and how we can effectively navigate between parents and children. Specifically, a concrete implementation of a tree must provide methods root, parent, num_children, children, __len__, and in the case of BinaryTree, the additional accessors left and right.

There are several choices for the internal representation of trees. We describe the most common representations in this section. We begin with the case of a ***binary tree***, since its shape is more narrowly defined.

8.3.1 Linked Structure for Binary Trees

A natural way to realize a binary tree T is to use a ***linked structure***, with a node (see Figure 8.11a) that maintains references to the element stored at a position p and to the nodes associated with the children and parent of p. If p is the root of T, then the parent field of p is None. Likewise, if p does not have a left child (respectively, right child), the associated field is None. The tree itself maintains an instance variable storing a reference to the root node (if any), and a variable, called size, that represents the overall number of nodes of T. We show such a linked structure representation of a binary tree in Figure 8.11b.

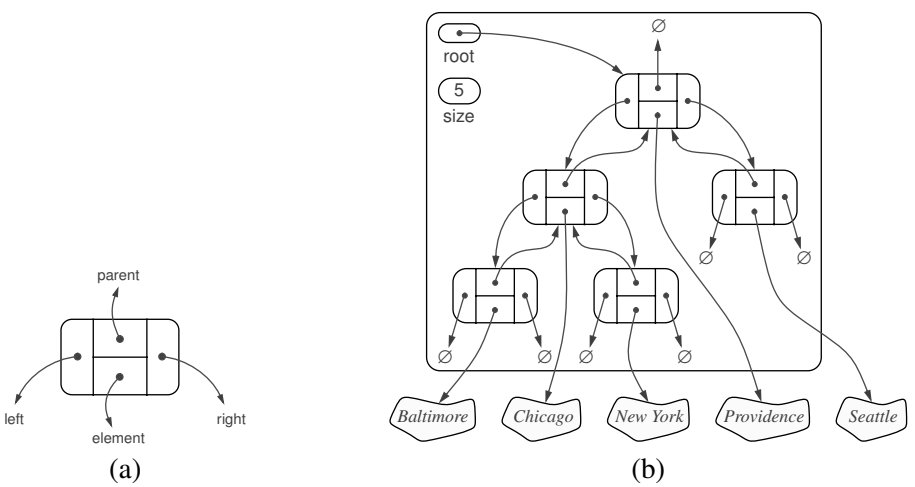


Figure 8.11: A linked structure for representing: (a) a single node; (b) a binary tree.

Python Implementation of a Linked Binary Tree Structure

In this section, we define a concrete LinkedBinaryTree class that implements the binary tree ADT by subclassing the BinaryTree class. Our general approach is very similar to what we used when developing the PositionalList in Section 7.4: We define a simple, nonpublic _Node class to represent a node, and a public Position class that wraps a node. We provide a _validate utility for robustly checking the validity of a given position instance when unwrapping it, and a _make_position utility for wrapping a node as a position to return to a caller.

Those definitions are provided in Code Fragment 8.8. As a formality, the new Position class is declared to inherit immediately from BinaryTree.Position. Technically, the BinaryTree class definition (see Code Fragment 8.7) does not formally declare such a nested class; it trivially inherits it from Tree.Position. A minor benefit from this design is that our position class inherits the __ne__ special method so that syntax p != q is derived appropriately relative to __eq__.

Our class definition continues, in Code Fragment 8.9, with a constructor and with concrete implementations for the methods that remain abstract in the Tree and BinaryTree classes. The constructor creates an empty tree by initializing _root to None and _size to zero. These accessor methods are implemented with careful use of the _validate and _make_position utilities to safeguard against boundary cases.

Operations for Updating a Linked Binary Tree

Thus far, we have provided functionality for examining an existing binary tree. However, the constructor for our LinkedBinaryTree class results in an empty tree and we have not provided any means for changing the structure or content of a tree.

We chose not to declare update methods as part of the Tree or BinaryTree abstract base classes for several reasons. First, although the principle of encapsulation suggests that the outward behaviors of a class need not depend on the internal representation, the *efficiency* of the operations depends greatly upon the representation. We prefer to have each concrete implementation of a tree class offer the most suitable options for updating a tree.

The second reason we do not provide update methods in the base class is that we may not want such update methods to be part of a public interface. There are many applications of trees, and some forms of update operations that are suitable for one application may be unacceptable in another. However, if we place an update method in a base class, any class that inherits from that base will inherit the update method. Consider, for example, the possibility of a method T.replace(p, e) that replaces the element stored at position p with another element e. Such a general method may be unacceptable in the context of an ***arithmetic expression tree*** (see Example 8.7 on page 312, and a later case study in Section 8.5), because we may want to enforce that internal nodes store only operators as elements.

For linked binary trees, a reasonable set of update methods to support for general usage are the following:

T.add_root(e): Create a root for an empty tree, storing e as the element, and return the position of that root; an error occurs if the tree is not empty.

T.add_left(p, e): Create a new node storing element e, link the node as the left child of position p, and return the resulting position; an error occurs if p already has a left child.

T.add_right(p, e): Create a new node storing element e, link the node as the right child of position p, and return the resulting position; an error occurs if p already has a right child.

T.replace(p, e): Replace the element stored at position p with element e, and return the previously stored element.

T.delete(p): Remove the node at position p, replacing it with its child, if any, and return the element that had been stored at p; an error occurs if p has two children.

T.attach(p, T1, T2): Attach the internal structure of trees T1 and T2, respectively, as the left and right subtrees of leaf position p of T, and reset T1 and T2 to empty trees; an error condition occurs if p is not a leaf.

We have specifically chosen this collection of operations because each can be implemented in $O(1)$ worst-case time with our linked representation. The most complex of these are delete and attach, due to the case analyses involving the various parent-child relationships and boundary conditions, yet there remains only a constant number of operations to perform. (The implementation of both methods could be greatly simplified if we used a tree representation with a sentinel node, akin to our treatment of positional lists; see Exercise C-8.40).

To avoid the problem of undesirable update methods being inherited by subclasses of LinkedBinaryTree, we have chosen an implementation in which none of the above methods are publicly supported. Instead, we provide *nonpublic* versions of each, for example, providing the underscored _delete in lieu of a public delete. Our implementations of these six update methods are provided in Code Fragments 8.10 and 8.11.

In particular applications, subclasses of LinkedBinaryTree can invoke the nonpublic methods internally, while preserving a public interface that is appropriate for the application. A subclass may also choose to wrap one or more of the nonpublic update methods with a public method to expose it to the user. We leave as an exercise (R-8.15), the task of defining a MutableLinkedBinaryTree subclass that provides public methods wrapping each of these six update methods.

```
class LinkedBinaryTree(BinaryTree):
  """Linked representation of a binary tree structure."""

  class _Node:          # Lightweight, nonpublic class for storing a node.
    __slots__ = '_element', '_parent', '_left', '_right'
    def __init__(self, element, parent=None, left=None, right=None):
      self._element = element
      self._parent = parent
      self._left = left
      self._right = right

  class Position(BinaryTree.Position):
    """An abstraction representing the location of a single element."""

    def __init__(self, container, node):
      """Constructor should not be invoked by user."""
      self._container = container
      self._node = node

    def element(self):
      """Return the element stored at this Position."""
      return self._node._element

    def __eq__(self, other):
      """Return True if other is a Position representing the same location."""
      return type(other) is type(self) and other._node is self._node

  def _validate(self, p):
    """Return associated node, if position is valid."""
    if not isinstance(p, self.Position):
      raise TypeError('p must be proper Position type')
    if p._container is not self:
      raise ValueError('p does not belong to this container')
    if p._node._parent is p._node:          # convention for deprecated nodes
      raise ValueError('p is no longer valid')
    return p._node

  def _make_position(self, node):
    """Return Position instance for given node (or None if no node)."""
    return self.Position(self, node) if node is not None else None
```

Code Fragment 8.8: The beginning of our LinkedBinaryTree class (continued in Code Fragments 8.9 through 8.11).

```
41    #---------------------------- binary tree constructor ----------------------------
42    def __init__(self):
43      """Create an initially empty binary tree."""
44      self._root = None
45      self._size = 0
46
47    #---------------------------- public accessors ----------------------------
48    def __len__(self):
49      """Return the total number of elements in the tree."""
50      return self._size
51
52    def root(self):
53      """Return the root Position of the tree (or None if tree is empty)."""
54      return self._make_position(self._root)
55
56    def parent(self, p):
57      """Return the Position of p's parent (or None if p is root)."""
58      node = self._validate(p)
59      return self._make_position(node._parent)
60
61    def left(self, p):
62      """Return the Position of p's left child (or None if no left child)."""
63      node = self._validate(p)
64      return self._make_position(node._left)
65
66    def right(self, p):
67      """Return the Position of p's right child (or None if no right child)."""
68      node = self._validate(p)
69      return self._make_position(node._right)
70
71    def num_children(self, p):
72      """Return the number of children of Position p."""
73      node = self._validate(p)
74      count = 0
75      if node._left is not None:          # left child exists
76        count += 1
77      if node._right is not None:         # right child exists
78        count += 1
79      return count
```

Code Fragment 8.9: Public accessors for our LinkedBinaryTree class. The class begins in Code Fragment 8.8 and continues in Code Fragments 8.10 and 8.11.

```
80    def _add_root(self, e):
81      """Place element e at the root of an empty tree and return new Position.
82
83      Raise ValueError if tree nonempty.
84      """
85      if self._root is not None: raise ValueError('Root exists')
86      self._size = 1
87      self._root = self._Node(e)
88      return self._make_position(self._root)
89
90    def _add_left(self, p, e):
91      """Create a new left child for Position p, storing element e.
92
93      Return the Position of new node.
94      Raise ValueError if Position p is invalid or p already has a left child.
95      """
96      node = self._validate(p)
97      if node._left is not None: raise ValueError('Left child exists')
98      self._size += 1
99      node._left = self._Node(e, node)                    # node is its parent
100     return self._make_position(node._left)
101
102   def _add_right(self, p, e):
103     """Create a new right child for Position p, storing element e.
104
105     Return the Position of new node.
106     Raise ValueError if Position p is invalid or p already has a right child.
107     """
108     node = self._validate(p)
109     if node._right is not None: raise ValueError('Right child exists')
110     self._size += 1
111     node._right = self._Node(e, node)                   # node is its parent
112     return self._make_position(node._right)
113
114   def _replace(self, p, e):
115     """Replace the element at position p with e, and return old element."""
116     node = self._validate(p)
117     old = node._element
118     node._element = e
119     return old
```

Code Fragment 8.10: Nonpublic update methods for the LinkedBinaryTree class (continued in Code Fragment 8.11).

```
120   def _delete(self, p):
121     """Delete the node at Position p, and replace it with its child, if any.
122
123     Return the element that had been stored at Position p.
124     Raise ValueError if Position p is invalid or p has two children.
125     """
126     node = self._validate(p)
127     if self.num_children(p) == 2: raise ValueError('p has two children')
128     child = node._left if node._left else node._right        # might be None
129     if child is not None:
130       child._parent = node._parent      # child's grandparent becomes parent
131     if node is self._root:
132       self._root = child                # child becomes root
133     else:
134       parent = node._parent
135       if node is parent._left:
136         parent._left = child
137       else:
138         parent._right = child
139     self._size -= 1
140     node._parent = node                 # convention for deprecated node
141     return node._element
142
143   def _attach(self, p, t1, t2):
144     """Attach trees t1 and t2 as left and right subtrees of external p."""
145     node = self._validate(p)
146     if not self.is_leaf(p): raise ValueError('position must be leaf')
147     if not type(self) is type(t1) is type(t2):   # all 3 trees must be same type
148       raise TypeError('Tree types must match')
149     self._size += len(t1) + len(t2)
150     if not t1.is_empty():               # attached t1 as left subtree of node
151       t1._root._parent = node
152       node._left = t1._root
153       t1._root = None                   # set t1 instance to empty
154       t1._size = 0
155     if not t2.is_empty():               # attached t2 as right subtree of node
156       t2._root._parent = node
157       node._right = t2._root
158       t2._root = None                   # set t2 instance to empty
159       t2._size = 0
```

Code Fragment 8.11: Nonpublic update methods for the LinkedBinaryTree class (continued from Code Fragment 8.10).

Performance of the Linked Binary Tree Implementation

To summarize the efficiencies of the linked structure representation, we analyze the running times of the LinkedBinaryTree methods, including derived methods that are inherited from the Tree and BinaryTree classes:

- The len method, implemented in LinkedBinaryTree, uses an instance variable storing the number of nodes of T and takes $O(1)$ time. Method is_empty, inherited from Tree, relies on a single call to len and thus takes $O(1)$ time.
- The accessor methods root, left, right, parent, and num_children are implemented directly in LinkedBinaryTree and take $O(1)$ time. The sibling and children methods are derived in BinaryTree based on a constant number of calls to these other accessors, so they run in $O(1)$ time as well.
- The is_root and is_leaf methods, from the Tree class, both run in $O(1)$ time, as is_root calls root and then relies on equivalence testing of positions, while is_leaf calls left and right and verifies that None is returned by both.
- Methods depth and height were each analyzed in Section 8.1.3. The depth method at position p runs in $O(d_p + 1)$ time where d_p is its depth; the height method on the root of the tree runs in $O(n)$ time.
- The various update methods add_root, add_left, add_right, replace, delete, and attach (that is, their nonpublic implementations) each run in $O(1)$ time, as they involve relinking only a constant number of nodes per operation.

Table 8.1 summarizes the performance of the linked structure implementation of a binary tree.

Operation	**Running Time**
len, is_empty	$O(1)$
root, parent, left, right, sibling, children, num_children	$O(1)$
is_root, is_leaf	$O(1)$
depth(p)	$O(d_p + 1)$
height	$O(n)$
add_root, add_left, add_right, replace, delete, attach	$O(1)$

Table 8.1: Running times for the methods of an n-node binary tree implemented with a linked structure. The space usage is $O(n)$.

8.3.2 Array-Based Representation of a Binary Tree

An alternative representation of a binary tree T is based on a way of numbering the positions of T. For every position p of T, let $f(p)$ be the integer defined as follows.

- If p is the root of T, then $f(p) = 0$.
- If p is the left child of position q, then $f(p) = 2f(q) + 1$.
- If p is the right child of position q, then $f(p) = 2f(q) + 2$.

The numbering function f is known as a ***level numbering*** of the positions in a binary tree T, for it numbers the positions on each level of T in increasing order from left to right. (See Figure 8.12.) Note well that the level numbering is based on *potential* positions within the tree, not actual positions of a given tree, so they are not necessarily consecutive. For example, in Figure 8.12(b), there are no nodes with level numbering 13 or 14, because the node with level numbering 6 has no children.

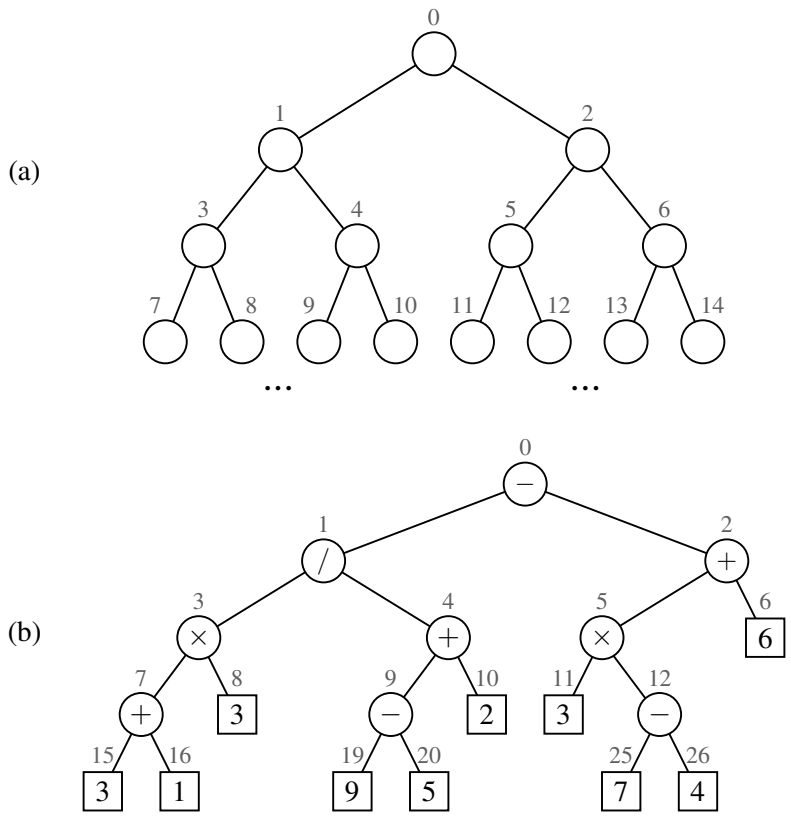


Figure 8.12: Binary tree level numbering: (a) general scheme; (b) an example.

The level numbering function f suggests a representation of a binary tree T by means of an array-based structure A (such as a Python list), with the element at position p of T stored at index $f(p)$ of the array. We show an example of an array-based representation of a binary tree in Figure 8.13.

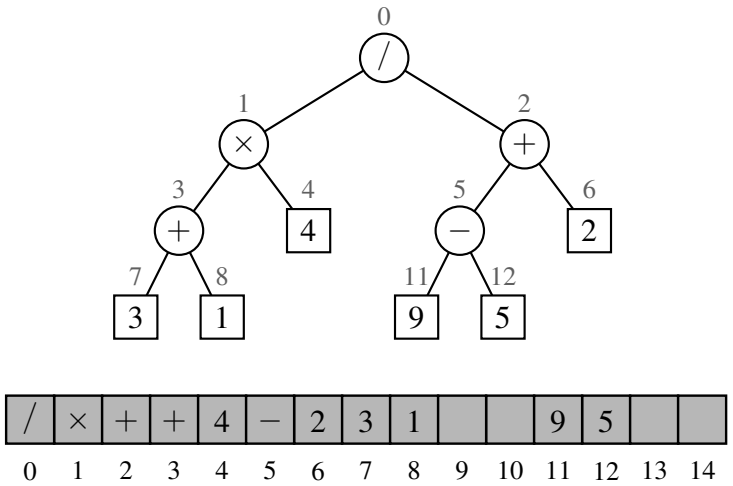


Figure 8.13: Representation of a binary tree by means of an array.

One advantage of an array-based representation of a binary tree is that a position p can be represented by the single integer $f(p)$, and that position-based methods such as root, parent, left, and right can be implemented using simple arithmetic operations on the number $f(p)$. Based on our formula for the level numbering, the left child of p has index $2f(p)+1$, the right child of p has index $2f(p)+2$, and the parent of p has index $\lfloor (f(p)-1)/2 \rfloor$. We leave the details of a complete implementation as an exercise (R-8.18).

The space usage of an array-based representation depends greatly on the shape of the tree. Let n be the number of nodes of T, and let f_M be the maximum value of $f(p)$ over all the nodes of T. The array A requires length $N = 1 + f_M$, since elements range from $A[0]$ to $A[f_M]$. Note that A may have a number of empty cells that do not refer to existing nodes of T. In fact, in the worst case, $N = 2^n - 1$, the justification of which is left as an exercise (R-8.16). In Section 9.3, we will see a class of binary trees, called "heaps" for which $N = n$. Thus, in spite of the worst-case space usage, there are applications for which the array representation of a binary tree is space efficient. Still, for general binary trees, the exponential worst-case space requirement of this representation is prohibitive.

Another drawback of an array representation is that some update operations for trees cannot be efficiently supported. For example, deleting a node and promoting its child takes $O(n)$ time because it is not just the child that moves locations within the array, but all descendants of that child.

8.3.3 Linked Structure for General Trees

When representing a binary tree with a linked structure, each node explicitly maintains fields left and right as references to individual children. For a general tree, there is no a priori limit on the number of children that a node may have. A natural way to realize a general tree T as a linked structure is to have each node store a single *container* of references to its children. For example, a children field of a node can be a Python list of references to the children of the node (if any). Such a linked representation is schematically illustrated in Figure 8.14.

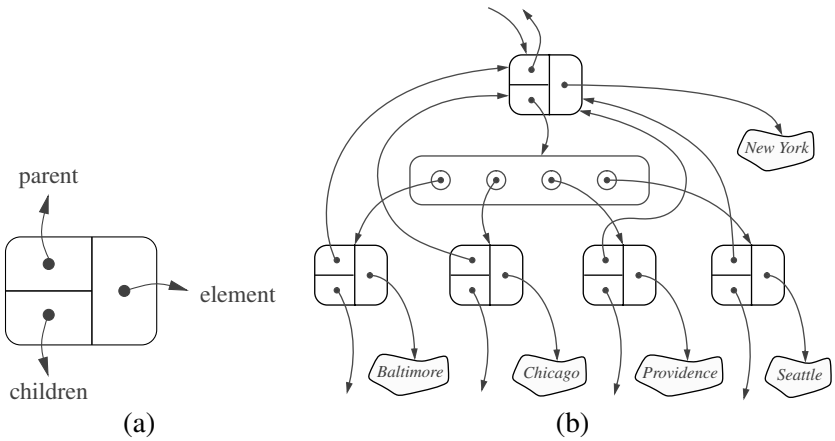


Figure 8.14: The linked structure for a general tree: (a) the structure of a node; (b) a larger portion of the data structure associated with a node and its children.

Table 8.2 summarizes the performance of the implementation of a general tree using a linked structure. The analysis is left as an exercise (R-8.14), but we note that, by using a collection to store the children of each position p, we can implement children(p) by simply iterating that collection.

Operation	Running Time
len, is_empty	$O(1)$
root, parent, is_root, is_leaf	$O(1)$
children(p)	$O(c_p+1)$
depth(p)	$O(d_p+1)$
height	$O(n)$

Table 8.2: Running times of the accessor methods of an n-node general tree implemented with a linked structure. We let c_p denote the number of children of a position p. The space usage is $O(n)$.

8.4 Tree Traversal Algorithms

A ***traversal*** of a tree T is a systematic way of accessing, or "visiting," all the positions of T. The specific action associated with the "visit" of a position p depends on the application of this traversal, and could involve anything from incrementing a counter to performing some complex computation for p. In this section, we describe several common traversal schemes for trees, implement them in the context of our various tree classes, and discuss several common applications of tree traversals.

8.4.1 Preorder and Postorder Traversals of General Trees

In a ***preorder traversal*** of a tree T, the root of T is visited first and then the subtrees rooted at its children are traversed recursively. If the tree is ordered, then the subtrees are traversed according to the order of the children. The pseudo-code for the preorder traversal of the subtree rooted at a position p is shown in Code Fragment 8.12.

```
Algorithm preorder(T, p):
  perform the "visit" action for position p
  for each child c in T.children(p) do
    preorder(T, c)                {recursively traverse the subtree rooted at c}
```

Code Fragment 8.12: Algorithm preorder for performing the preorder traversal of a subtree rooted at position p of a tree T.

Figure 8.15 portrays the order in which positions of a sample tree are visited during an application of the preorder traversal algorithm.

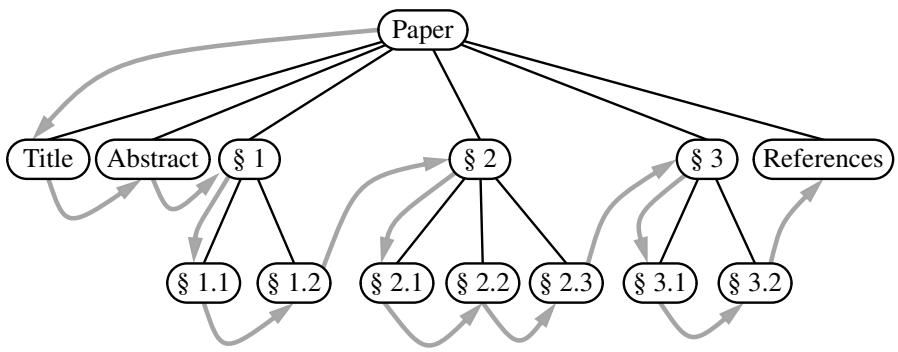


Figure 8.15: Preorder traversal of an ordered tree, where the children of each position are ordered from left to right.

Postorder Traversal

Another important tree traversal algorithm is the ***postorder traversal***. In some sense, this algorithm can be viewed as the opposite of the preorder traversal, because it recursively traverses the subtrees rooted at the children of the root first, and then visits the root (hence, the name "postorder"). Pseudo-code for the postorder traversal is given in Code Fragment 8.13, and an example of a postorder traversal is portrayed in Figure 8.16.

```
Algorithm postorder(T, p):
  for each child c in T.children(p) do
    postorder(T, c)                    {recursively traverse the subtree rooted at c}
  perform the "visit" action for position p
```

Code Fragment 8.13: Algorithm postorder for performing the postorder traversal of a subtree rooted at position p of a tree T.

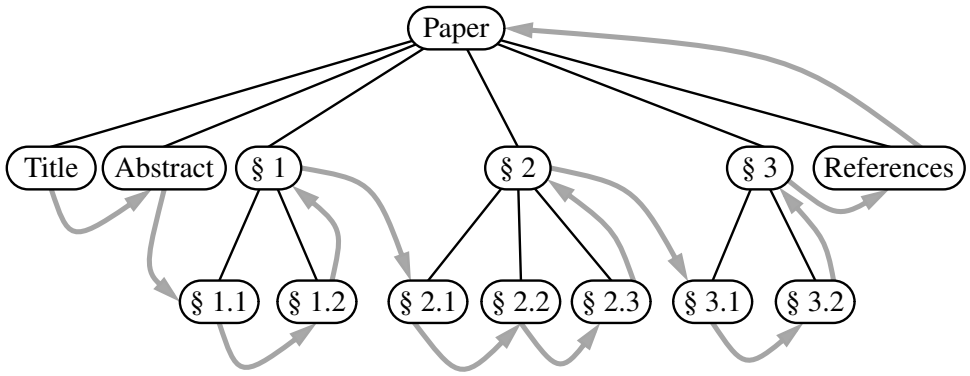


Figure 8.16: Postorder traversal of the ordered tree of Figure 8.15.

Running-Time Analysis

Both preorder and postorder traversal algorithms are efficient ways to access all the positions of a tree. The analysis of either of these traversal algorithms is similar to that of algorithm height2, given in Code Fragment 8.5 of Section 8.1.3. At each position p, the nonrecursive part of the traversal algorithm requires time $O(c_p + 1)$, where c_p is the number of children of p, under the assumption that the "visit" itself takes $O(1)$ time. By Proposition 8.5, the overall running time for the traversal of tree T is $O(n)$, where n is the number of positions in the tree. This running time is asymptotically optimal since the traversal must visit all the n positions of the tree.

8.4.2 Breadth-First Tree Traversal

Although the preorder and postorder traversals are common ways of visiting the positions of a tree, another common approach is to traverse a tree so that we visit all the positions at depth d before we visit the positions at depth $d+1$. Such an algorithm is known as a ***breadth-first traversal***.

A breadth-first traversal is a common approach used in software for playing games. A ***game tree*** represents the possible choices of moves that might be made by a player (or computer) during a game, with the root of the tree being the initial configuration for the game. For example, Figure 8.17 displays a partial game tree for Tic-Tac-Toe.

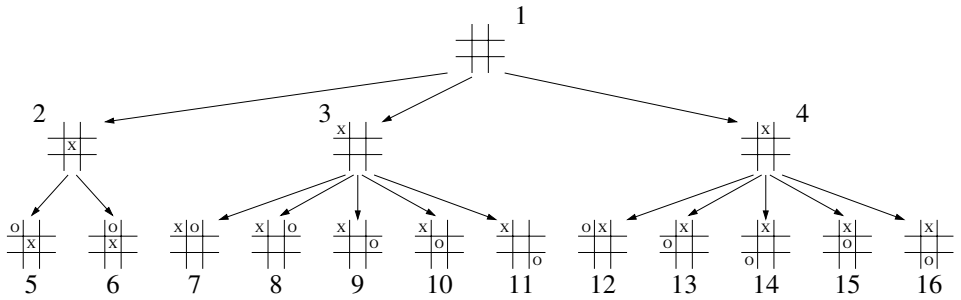


Figure 8.17: Partial game tree for Tic-Tac-Toe, with annotations displaying the order in which positions are visited in a breadth-first traversal.

A breadth-first traversal of such a game tree is often performed because a computer may be unable to explore a complete game tree in a limited amount of time. So the computer will consider all moves, then responses to those moves, going as deep as computational time allows.

Pseudo-code for a breadth-first traversal is given in Code Fragment 8.14. The process is not recursive, since we are not traversing entire subtrees at once. We use a queue to produce a FIFO (i.e., first-in first-out) semantics for the order in which we visit nodes. The overall running time is $O(n)$, due to the n calls to enqueue and n calls to dequeue.

```
Algorithm breadthfirst(T):
  Initialize queue Q to contain T.root( )
  while Q not empty do
    p = Q.dequeue( )                    {p is the oldest entry in the queue}
    perform the "visit" action for position p
    for each child c in T.children(p) do
      Q.enqueue(c)     {add p's children to the end of the queue for later visits}
```

Code Fragment 8.14: Algorithm for performing a breadth-first traversal of a tree.

8.4.3 Inorder Traversal of a Binary Tree

The standard preorder, postorder, and breadth-first traversals that were introduced for general trees, can be directly applied to binary trees. In this section, we introduce another common traversal algorithm specifically for a binary tree.

During an ***inorder traversal***, we visit a position between the recursive traversals of its left and right subtrees. The inorder traversal of a binary tree T can be informally viewed as visiting the nodes of T "from left to right." Indeed, for every position p, the inorder traversal visits p after all the positions in the left subtree of p and before all the positions in the right subtree of p. Pseudo-code for the inorder traversal algorithm is given in Code Fragment 8.15, and an example of an inorder traversal is portrayed in Figure 8.18.

```
Algorithm inorder(p):
  if p has a left child lc then
    inorder(lc)                    {recursively traverse the left subtree of p}
  perform the "visit" action for position p
  if p has a right child rc then
    inorder(rc)                    {recursively traverse the right subtree of p}
```

Code Fragment 8.15: Algorithm inorder for performing an inorder traversal of a subtree rooted at position p of a binary tree.

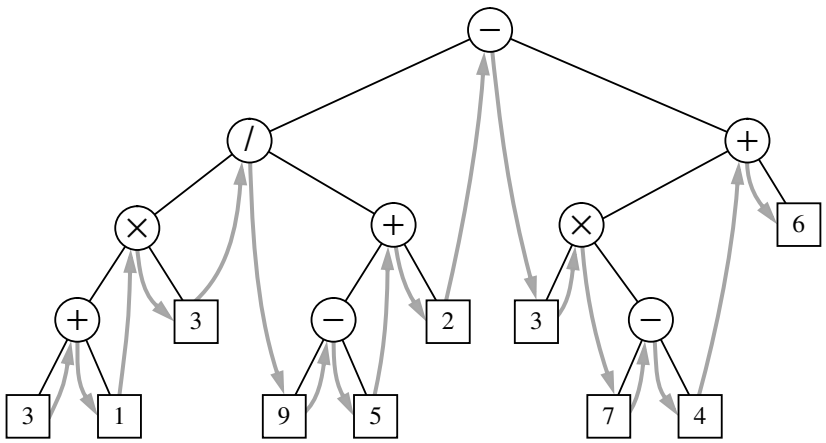


Figure 8.18: Inorder traversal of a binary tree.

The inorder traversal algorithm has several important applications. When using a binary tree to represent an arithmetic expression, as in Figure 8.18, the inorder traversal visits positions in a consistent order with the standard representation of the expression, as in $3+1\times 3/9-5+2\ldots$ (albeit without parentheses).

Binary Search Trees

An important application of the inorder traversal algorithm arises when we store an ordered sequence of elements in a binary tree, defining a structure we call a ***binary search tree***. Let S be a set whose unique elements have an order relation. For example, S could be a set of integers. A binary search tree for S is a binary tree T such that, for each position p of T:

- Position p stores an element of S, denoted as $e(p)$.
- Elements stored in the left subtree of p (if any) are less than $e(p)$.
- Elements stored in the right subtree of p (if any) are greater than $e(p)$.

An example of a binary search tree is shown in Figure 8.19. The above properties assure that an inorder traversal of a binary search tree T visits the elements in nondecreasing order.

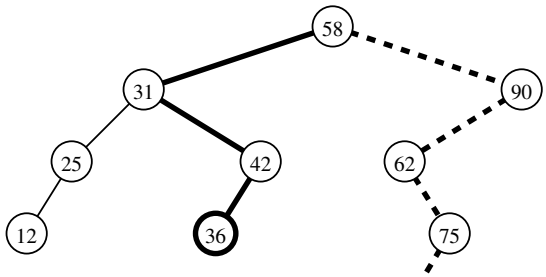


Figure 8.19: A binary search tree storing integers. The solid path is traversed when searching (successfully) for 36. The dashed path is traversed when searching (unsuccessfully) for 70.

We can use a binary search tree T for set S to find whether a given search value v is in S, by traversing a path down the tree T, starting at the root. At each internal position p encountered, we compare our search value v with the element $e(p)$ stored at p. If $v < e(p)$, then the search continues in the left subtree of p. If $v = e(p)$, then the search terminates successfully. If $v > e(p)$, then the search continues in the right subtree of p. Finally, if we reach an empty subtree, the search terminates unsuccessfully. In other words, a binary search tree can be viewed as a binary decision tree (recall Example 8.6), where the question asked at each internal node is whether the element at that node is less than, equal to, or larger than the element being searched for. We illustrate several examples of the search operation in Figure 8.19.

Note that the running time of searching in a binary search tree T is proportional to the height of T. Recall from Proposition 8.8 that the height of a binary tree with n nodes can be as small as $\log(n+1)-1$ or as large as $n-1$. Thus, binary search trees are most efficient when they have small height. Chapter 11 is devoted to the study of search trees.

8.4.4 Implementing Tree Traversals in Python

When first defining the tree ADT in Section 8.1.2, we stated that tree T should include support for the following methods:

T.positions( ): Generate an iteration of all *positions* of tree T.

iter(T): Generate an iteration of all *elements* stored within tree T.

At that time, we did not make any assumption about the order in which these iterations report their results. In this section, we demonstrate how any of the tree traversal algorithms we have introduced could be used to produce these iterations.

To begin, we note that it is easy to produce an iteration of all elements of a tree, if we rely on a presumed iteration of all positions. Therefore, support for the iter(T) syntax can be formally provided by a concrete implementation of the special method __iter__ within the abstract base class Tree. We rely on Python's generator syntax as the mechanism for producing iterations. (See Section 1.8.) Our implementation of Tree.__iter__ is given in Code Fragment 8.16.

```
75  def __iter__(self):
76    """Generate an iteration of the tree's elements."""
77    for p in self.positions( ):        # use same order as positions()
78      yield p.element( )               # but yield each element
```

Code Fragment 8.16: Iterating all elements of a Tree instance, based upon an iteration of the positions of the tree. This code should be included in the body of the Tree class.

To implement the positions method, we have a choice of tree traversal algorithms. Given that there are advantages to each of those traversal orders, we will provide independent implementations of each strategy that can be called directly by a user of our class. We can then trivially adapt one of those as a default order for the positions method of the tree ADT.

Preorder Traversal

We begin by considering the ***preorder traversal*** algorithm. We will support a public method with calling signature T.preorder( ) for tree T, which generates a preorder iteration of all positions within the tree. However, the recursive algorithm for generating a preorder traversal, as originally described in Code Fragment 8.12, must be parameterized by a specific position within the tree that serves as the root of a subtree to traverse. A standard solution for such a circumstance is to define a nonpublic utility method with the desired recursive parameterization, and then to have the public method preorder invoke the nonpublic method upon the root of the tree. Our implementation of such a design is given in Code Fragment 8.17.

```
79   def preorder(self):
80     """Generate a preorder iteration of positions in the tree."""
81     if not self.is_empty( ):
82       for p in self._subtree_preorder(self.root( )):       # start recursion
83         yield p
84
85   def _subtree_preorder(self, p):
86     """Generate a preorder iteration of positions in subtree rooted at p."""
87     yield p                                    # visit p before its subtrees
88     for c in self.children(p):                 # for each child c
89       for other in self._subtree_preorder(c):  # do preorder of c's subtree
90         yield other                            # yielding each to our caller
```

Code Fragment 8.17: Support for performing a preorder traversal of a tree. This code should be included in the body of the Tree class.

Formally, both preorder and the utility _subtree_preorder are generators. Rather than perform a "visit" action from within this code, we yield each position to the caller and let the caller decide what action to perform at that position.

The _subtree_preorder method is the recursive one. However, because we are relying on generators rather than traditional functions, the recursion has a slightly different form. In order to yield all positions within the subtree of child c, we loop over the positions yielded by the recursive call self._subtree_preorder(c), and re-yield each position in the outer context. Note that if p is a leaf, the for loop over self.children(p) is trivial (this is the base case for our recursion).

We rely on a similar technique in the public preorder method to re-yield all positions that are generated by the recursive process starting at the root of the tree; if the tree is empty, nothing is yielded. At this point, we have provided full support for the preorder generator. A user of the class can therefore write code such as

```
for p in T.preorder( ):
  # "visit" position p
```

The official tree ADT requires that all trees support a positions method as well. To use a preorder traversal as the default order of iteration, we include the definition shown in Code Fragment 8.18 within our Tree class. Rather than loop over the results returned by the preorder call, we return the entire iteration as an object.

```
91   def positions(self):
92     """Generate an iteration of the tree's positions."""
93     return self.preorder( )              # return entire preorder iteration
```

Code Fragment 8.18: An implementation of the positions method for the Tree class that relies on a preorder traversal to generate the results.

Postorder Traversal

We can implement a postorder traversal using very similar technique as with a preorder traversal. The only difference is that within the recursive utility for a postorder we wait to yield position p until *after* we have recursively yield the positions in its subtrees. An implementation is given in Code Fragment 8.19.

```
94   def postorder(self):
95     """Generate a postorder iteration of positions in the tree."""
96     if not self.is_empty( ):
97       for p in self._subtree_postorder(self.root( )):       # start recursion
98         yield p
99
100  def _subtree_postorder(self, p):
101    """Generate a postorder iteration of positions in subtree rooted at p."""
102    for c in self.children(p):                    # for each child c
103      for other in self._subtree_postorder(c):    # do postorder of c's subtree
104        yield other                               # yielding each to our caller
105    yield p                                       # visit p after its subtrees
```

Code Fragment 8.19: Support for performing a postorder traversal of a tree. This code should be included in the body of the Tree class.

Breadth-First Traversal

In Code Fragment 8.20, we provide an implementation of the breadth-first traversal algorithm in the context of our Tree class. Recall that the breadth-first traversal algorithm is not recursive; it relies on a queue of positions to manage the traversal process. Our implementation uses the LinkedQueue class from Section 7.1.2, although any implementation of the queue ADT would suffice.

Inorder Traversal for Binary Trees

The preorder, postorder, and breadth-first traversal algorithms are applicable to all trees, and so we include their implementations within the Tree abstract base class. Those methods are inherited by the abstract BinaryTree class, the concrete LinkedBinaryTree class, and any other dependent tree classes we might develop.

The inorder traversal algorithm, because it explicitly relies on the notion of a left and right child of a node, only applies to binary trees. We therefore include its definition within the body of the BinaryTree class. We use a similar technique to implement an inorder traversal (Code Fragment 8.21) as we did with preorder and postorder traversals.

```
106   def breadthfirst(self):
107     """Generate a breadth-first iteration of the positions of the tree."""
108     if not self.is_empty( ):
109       fringe = LinkedQueue( )                # known positions not yet yielded
110       fringe.enqueue(self.root( ))           # starting with the root
111       while not fringe.is_empty( ):
112         p = fringe.dequeue( )                # remove from front of the queue
113         yield p                              # report this position
114         for c in self.children(p):
115           fringe.enqueue(c)                  # add children to back of queue
```

Code Fragment 8.20: An implementation of a breadth-first traversal of a tree. This code should be included in the body of the Tree class.

```
37   def inorder(self):
38     """Generate an inorder iteration of positions in the tree."""
39     if not self.is_empty( ):
40       for p in self._subtree_inorder(self.root( )):
41         yield p
42
43   def _subtree_inorder(self, p):
44     """Generate an inorder iteration of positions in subtree rooted at p."""
45     if self.left(p) is not None:          # if left child exists, traverse its subtree
46       for other in self._subtree_inorder(self.left(p)):
47         yield other
48     yield p                               # visit p between its subtrees
49     if self.right(p) is not None:         # if right child exists, traverse its subtree
50       for other in self._subtree_inorder(self.right(p)):
51         yield other
```

Code Fragment 8.21: Support for performing an inorder traversal of a binary tree. This code should be included in the BinaryTree class (given in Code Fragment 8.7).

For many applications of binary trees, an inorder traversal provides a natural iteration. We could make it the default for the BinaryTree class by overriding the positions method that was inherited from the Tree class (see Code Fragment 8.22).

```
52   # override inherited version to make inorder the default
53   def positions(self):
54     """Generate an iteration of the tree's positions."""
55     return self.inorder( )                # make inorder the default
```

Code Fragment 8.22: Defining the BinaryTree.position method so that positions are reported using inorder traversal.

8.4.5 Applications of Tree Traversals

In this section, we demonstrate several representative applications of tree traversals, including some customizations of the standard traversal algorithms.

Table of Contents

When using a tree to represent the hierarchical structure of a document, a preorder traversal of the tree can naturally be used to produce a table of contents for the document. For example, the table of contents associated with the tree from Figure 8.15 is displayed in Figure 8.20. Part (a) of that figure gives a simple presentation with one element per line; part (b) shows a more attractive presentation produced by indenting each element based on its depth within the tree. A similar presentation could be used to display the contents of a computer's file system, based on its tree representation (as in Figure 8.3).

```
Paper                Paper
Title                  Title
Abstract               Abstract
§1                     §1
§1.1                     §1.1
§1.2                     §1.2
§2                     §2
§2.1                     §2.1
...                      ...

(a)                  (b)
```

Figure 8.20: Table of contents for a document represented by the tree in Figure 8.15: (a) without indentation; (b) with indentation based on depth within the tree.

The unindented version of the table of contents, given a tree T, can be produced with the following code:

```
for p in T.preorder( ):
  print(p.element( ))
```

To produce the presentation of Figure 8.20(b), we indent each element with a number of spaces equal to twice the element's depth in the tree (hence, the root element was unindented). Although we could replace the body of the above loop with the statement `print(2*T.depth(p)*' ' + str(p.element( )))`, such an approach is unnecessarily inefficient. Although the work to produce the preorder traversal runs in $O(n)$ time, based on the analysis of Section 8.4.1, the calls to `depth` incur a hidden cost. Making a call to `depth` from every position of the tree results in $O(n^2)$ worst-case time, as noted when analyzing the algorithm `height1` in Section 8.1.3.

A preferred approach to producing an indented table of contents is to redesign a top-down recursion that includes the current depth as an additional parameter. Such an implementation is provided in Code Fragment 8.23. This implementation runs in worst-case $O(n)$ time (except, technically, the time it takes to print strings of increasing lengths).

```
1  def preorder_indent(T, p, d):
2    """Print preorder representation of subtree of T rooted at p at depth d."""
3    print(2*d*' ' + str(p.element( )))          # use depth for indentation
4    for c in T.children(p):
5      preorder_indent(T, c, d+1)                # child depth is d+1
```

Code Fragment 8.23: Efficient recursion for printing indented version of a preorder traversal. On a complete tree T, the recursion should be started with form preorder_indent(T, T.root( ), 0).

In the example of Figure 8.20, we were fortunate in that the numbering was embedded within the elements of the tree. More generally, we might be interested in using a preorder traversal to display the structure of a tree, with indentation and also explicit numbering that was not present in the tree. For example, we might display the tree from Figure 8.2 beginning as:

```
Electronics R'Us
  1 R&D
  2 Sales
    2.1 Domestic
    2.2 International
      2.2.1 Canada
      2.2.2 S. America
```

This is more challenging, because the numbers used as labels are implicit in the structure of the tree. A label depends on the index of each position, relative to its siblings, along the path from the root to the current position. To accomplish the task, we add a representation of that path as an additional parameter to the recursive signature. Specifically, we use a list of zero-indexed numbers, one for each position along the downward path, other than the root. (We convert those numbers to one-indexed form when printing.)

At the implementation level, we wish to avoid the inefficiency of duplicating such lists when sending a new parameter from one level of the recursion to the next. A standard solution is to share the same list instance throughout the recursion. At one level of the recursion, a new entry is temporarily added to the end of the list before making further recursive calls. In order to "leave no trace," that same block of code must remove the extraneous entry from the list before completing its task. An implementation based on this approach is given in Code Fragment 8.24.

```
1  def preorder_label(T, p, d, path):
2    """Print labeled representation of subtree of T rooted at p at depth d."""
3    label = '.'.join(str(j+1) for j in path)    # displayed labels are one-indexed
4    print(2*d*'  ' + label, p.element())
5    path.append(0)                              # path entries are zero-indexed
6    for c in T.children(p):
7      preorder_label(T, c, d+1, path)           # child depth is d+1
8      path[-1] += 1
9    path.pop()
```

Code Fragment 8.24: Efficient recursion for printing an indented and *labeled* presentation of a preorder traversal.

Parenthetic Representations of a Tree

It is not possible to reconstruct a general tree, given only the preorder sequence of elements, as in Figure 8.20(a). Some additional context is necessary for the structure of the tree to be well defined. The use of indentation or numbered labels provides such context, with a very human-friendly presentation. However, there are more concise string representations of trees that are computer-friendly.

In this section, we explore one such representation. The ***parenthetic string representation*** $P(T)$ of tree T is recursively defined as follows. If T consists of a single position p, then

$$P(T) = \mathsf{str}(p.\mathsf{element}()).$$

Otherwise, it is defined recursively as,

$$P(T) = \mathsf{str}(p.\mathsf{element}()) + \texttt{'('} + P(T_1) + \texttt{', '} + \cdots + \texttt{', '} + P(T_k) + \texttt{')'}$$

where p is the root of T and $T_1, T_2, \ldots, T_k$ are the subtrees rooted at the children of p, which are given in order if T is an ordered tree. We are using "+" here to denote string concatenation. As an example, the parenthetic representation of the tree of Figure 8.2 would appear as follows (line breaks are cosmetic):

```
Electronics R'Us (R&D, Sales (Domestic, International (Canada,
S. America, Overseas (Africa, Europe, Asia, Australia))),
Purchasing, Manufacturing (TV, CD, Tuner))
```

Although the parenthetic representation is essentially a preorder traversal, we cannot easily produce the additional punctuation using the formal implementation of preorder, as given in Code Fragment 8.17. The opening parenthesis must be produced just before the loop over a position's children and the closing parenthesis must be produced just after that loop. Furthermore, the separating commas must be produced. The Python function parenthesize, shown in Code Fragment 8.25, is a custom traversal that prints such a parenthetic string representation of a tree T.

```
def parenthesize(T, p):
  """Print parenthesized representation of subtree of T rooted at p."""
  print(p.element( ), end='')          # use of end avoids trailing newline
  if not T.is_leaf(p):
    first_time = True
    for c in T.children(p):
      sep = ' (' if first_time else ', '          # determine proper separator
      print(sep, end='')
      first_time = False                # any future passes will not be the first
      parenthesize(T, c)                # recur on child
    print(')', end='')                  # include closing parenthesis
```

Code Fragment 8.25: Function that prints parenthetic string representation of a tree.

Computing Disk Space

In Example 8.1, we considered the use of a tree as a model for a file-system structure, with internal positions representing directories and leaves representing files. In fact, when introducing the use of recursion back in Chapter 4, we specifically examined the topic of file systems (see Section 4.1.4). Although we did not explicitly model it as a tree at that time, we gave an implementation of an algorithm for computing the disk usage (Code Fragment 4.5).

The recursive computation of disk space is emblematic of a *postorder* traversal, as we cannot effectively compute the total space used by a directory until *after* we know the space that is used by its children directories. Unfortunately, the formal implementation of postorder, as given in Code Fragment 8.19 does not suffice for this purpose. As it visits the position of a directory, there is no easy way to discern which of the previous positions represent children of that directory, nor how much recursive disk space was allocated.

We would like to have a mechanism for children to return information to the parent as part of the traversal process. A custom solution to the disk space problem, with each level of recursion providing a return value to the (parent) caller, is provided in Code Fragment 8.26.

```
def disk_space(T, p):
  """Return total disk space for subtree of T rooted at p."""
  subtotal = p.element( ).space( )          # space used at position p
  for c in T.children(p):
    subtotal += disk_space(T, c)            # add child's space to subtotal
  return subtotal
```

Code Fragment 8.26: Recursive computation of disk space for a tree. We assume that a space( ) method of each tree element reports the local space used at that position.

8.4.6 Euler Tours and the Template Method Pattern ⋆

The various applications described in Section 8.4.5 demonstrate the great power of recursive tree traversals. Unfortunately, they also show that the specific implementations of the preorder and postorder methods of our Tree class, or the inorder method of the BinaryTree class, are not general enough to capture the range of computations we desire. In some cases, we need more of a blending of the approaches, with initial work performed before recurring on subtrees, additional work performed after those recursions, and in the case of a binary tree, work performed between the two possible recursions. Furthermore, in some contexts it was important to know the depth of a position, or the complete path from the root to that position, or to return information from one level of the recursion to another. For each of the previous applications, we were able to develop a custom implementation to properly adapt the recursive ideas, but the great principles of object-oriented programming introduced in Section 2.1.1 include ***adaptability*** and ***reusability***.

In this section, we develop a more general framework for implementing tree traversals based on a concept known as an ***Euler tour traversal***. The Euler tour traversal of a general tree T can be informally defined as a "walk" around T, where we start by going from the root toward its leftmost child, viewing the edges of T as being "walls" that we always keep to our left. (See Figure 8.21.)

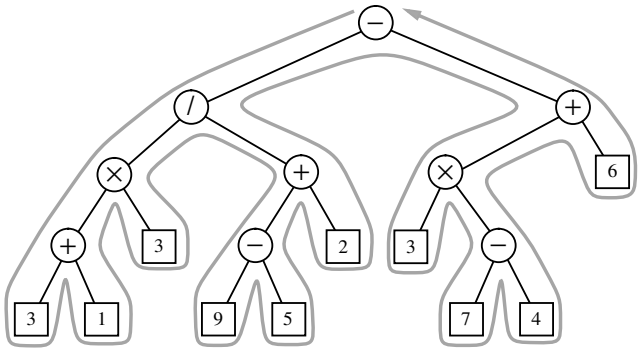


Figure 8.21: Euler tour traversal of a tree.

The complexity of the walk is $O(n)$, because it progresses exactly two times along each of the $n-1$ edges of the tree—once going downward along the edge, and later going upward along the edge. To unify the concept of preorder and postorder traversals, we can think of there being two notable "visits" to each position p:

- A "pre visit" occurs when first reaching the position, that is, when the walk passes immediately left of the node in our visualization.
- A "post visit" occurs when the walk later proceeds upward from that position, that is, when the walk passes to the right of the node in our visualization.

The process of an Euler tour can easily be viewed recursively. In between the "pre visit" and "post visit" of a given position will be a recursive tour of each of its subtrees. Looking at Figure 8.21 as an example, there is a contiguous portion of the entire tour that is itself an Euler tour of the subtree of the node with element "/". That tour contains two contiguous subtours, one traversing that position's left subtree and another traversing the right subtree. The pseudo-code for an Euler tour traversal of a subtree rooted at a position p is shown in Code Fragment 8.27.

```
Algorithm eulertour(T, p):
  perform the "pre visit" action for position p
  for each child c in T.children(p) do
    eulertour(T, c)                    {recursively tour the subtree rooted at c}
  perform the "post visit" action for position p
```

Code Fragment 8.27: Algorithm eulertour for performing an Euler tour traversal of a subtree rooted at position p of a tree.

The Template Method Pattern

To provide a framework that is reusable and adaptable, we rely on an interesting object-oriented software design pattern, the ***template method pattern***. The template method pattern describes a generic computation mechanism that can be specialized for a particular application by redefining certain steps. To allow customization, the primary algorithm calls auxiliary functions known as ***hooks*** at designated steps of the process.

In the context of an Euler tour traversal, we define two separate hooks, a previsit hook that is called before the subtrees are traversed, and a postvisit hook that is called after the completion of the subtree traversals. Our implementation will take the form of an EulerTour class that manages the process, and defines trivial definitions for the hooks that do nothing. The traversal can be customized by defining a subclass of EulerTour and overriding one or both hooks to provide specialized behavior.

Python Implementation

Our implementation of an EulerTour class is provided in Code Fragment 8.28. The primary recursive process is defined in the nonpublic _tour method. A tour instance is created by sending a reference to a specific tree to the constructor, and then by calling the public execute method, which beings the tour and returns a final result of the computation.

```
1  class EulerTour:
2    """Abstract base class for performing Euler tour of a tree.
3
4    _hook_previsit and _hook_postvisit may be overridden by subclasses.
5    """
6    def __init__(self, tree):
7      """Prepare an Euler tour template for given tree."""
8      self._tree = tree
9
10   def tree(self):
11     """Return reference to the tree being traversed."""
12     return self._tree
13
14   def execute(self):
15     """Perform the tour and return any result from post visit of root."""
16     if len(self._tree) > 0:
17       return self._tour(self._tree.root( ), 0, [ ])          # start the recursion
18
19   def _tour(self, p, d, path):
20     """Perform tour of subtree rooted at Position p.
21
22     p          Position of current node being visited
23     d          depth of p in the tree
24     path       list of indices of children on path from root to p
25     """
26     self._hook_previsit(p, d, path)                          # "pre visit" p
27     results = [ ]
28     path.append(0)             # add new index to end of path before recursion
29     for c in self._tree.children(p):
30       results.append(self._tour(c, d+1, path))      # recur on child's subtree
31       path[-1] += 1             # increment index
32     path.pop( )                 # remove extraneous index from end of path
33     answer = self._hook_postvisit(p, d, path, results)       # "post visit" p
34     return answer
35
36   def _hook_previsit(self, p, d, path):                      # can be overridden
37     pass
38
39   def _hook_postvisit(self, p, d, path, results):            # can be overridden
40     pass
```

Code Fragment 8.28: An EulerTour base class providing a framework for performing Euler tour traversals of a tree.

Based on our experience of customizing traversals for sample applications Section 8.4.5, we build support into the primary EulerTour for maintaining the recursive depth and the representation of the recursive path through a tree, using the approach that we introduced in Code Fragment 8.24. We also provide a mechanism for one recursive level to return a value to another when post-processing. Formally, our framework relies on the following two hooks that can be specialized:

- method _hook_previsit(p, d, path)
  This function is called once for each position, immediately before its subtrees (if any) are traversed. Parameter p is a position in the tree, d is the depth of that position, and path is a list of indices, using the convention described in the discussion of Code Fragment 8.24. No return value is expected from this function.
- method _hook_postvisit(p, d, path, results)
  This function is called once for each position, immediately after its subtrees (if any) are traversed. The first three parameters use the same convention as did _hook_previsit. The final parameter is a list of objects that were provided as return values from the post visits of the respective subtrees of p. Any value returned by this call will be available to the parent of p during its postvisit.

For more complex tasks, subclasses of EulerTour may also choose to initialize and maintain additional state in the form of instance variables that can be accessed within the bodies of the hooks.

Using the Euler Tour Framework

To demonstrate the flexibility of our Euler tour framework, we revisit the sample applications from Section 8.4.5. As a simple example, an indented preorder traversal, akin to that originally produced by Code Fragment 8.23, can be generated with the simple subclass given in Code Fragment 8.29.

```
1 class PreorderPrintIndentedTour(EulerTour):
2   def _hook_previsit(self, p, d, path):
3     print(2*d*' ' + str(p.element( )))
```

Code Fragment 8.29: A subclass of EulerTour that produces an indented preorder list of a tree's elements.

Such a tour would be started by creating an instance of the subclass for a given tree T, and invoking its execute method. This could be expressed as follows:

```
tour = PreorderPrintIndentedTour(T)
tour.execute( )
```

A labeled version of an indented, preorder presentation, akin to Code Fragment 8.24, could be generated by the new subclass of EulerTour shown in Code Fragment 8.30.

```
1  class PreorderPrintIndentedLabeledTour(EulerTour):
2    def _hook_previsit(self, p, d, path):
3      label = '.'.join(str(j+1) for j in path)    # labels are one-indexed
4      print(2*d*' ' + label, p.element( ))
```

Code Fragment 8.30: A subclass of EulerTour that produces a labeled and indented, preorder list of a tree's elements.

To produce the parenthetic string representation, originally achieved with Code Fragment 8.25, we define a subclass that overrides both the previsit and postvisit hooks. Our new implementation is given in Code Fragment 8.31.

```
1  class ParenthesizeTour(EulerTour):
2    def _hook_previsit(self, p, d, path):
3      if path and path[-1] > 0:                   # p follows a sibling
4        print(', ', end='')                       # so preface with comma
5      print(p.element( ), end='')                 # then print element
6      if not self.tree( ).is_leaf(p):             # if p has children
7        print(' (', end='')                       # print opening parenthesis
8
9    def _hook_postvisit(self, p, d, path, results):
10     if not self.tree( ).is_leaf(p):             # if p has children
11       print(')', end='')                        # print closing parenthesis
```

Code Fragment 8.31: A subclass of EulerTour that prints a parenthetic string representation of a tree.

Notice that in this implementation, we need to invoke a method on the tree instance that is being traversed from within the hooks. The public tree( ) method of the EulerTour class serves as an accessor for that tree.

Finally, the task of computing disk space, as originally implemented in Code Fragment 8.26, can be performed quite easily with the EulerTour subclass shown in Code Fragment 8.32. The postvisit result of the root will be returned by the call to execute( ).

```
1  class DiskSpaceTour(EulerTour):
2    def _hook_postvisit(self, p, d, path, results):
3      # we simply add space associated with p to that of its subtrees
4      return p.element( ).space( ) + sum(results)
```

Code Fragment 8.32: A subclass of EulerTour that computes disk space for a tree.

The Euler Tour Traversal of a Binary Tree

In Section 8.4.6, we introduced the concept of an Euler tour traversal of a general graph, using the template method pattern in designing the EulerTour class. That class provided methods _hook_previsit and _hook_postvisit that could be overridden to customize a tour. In Code Fragment 8.33 we provide a BinaryEulerTour specialization that includes an additional _hook_invisit that is called once for each position—after its left subtree is traversed, but before its right subtree is traversed.

Our implementation of BinaryEulerTour replaces the original _tour utility to specialize to the case in which a node has at most two children. If a node has only one child, a tour differentiates between whether that is a left child or a right child, with the "in visit" taking place after the visit of a sole left child, but before the visit of a sole right child. In the case of a leaf, the three hooks are called in succession.

```
1  class BinaryEulerTour(EulerTour):
2    """Abstract base class for performing Euler tour of a binary tree.
3
4    This version includes an additional _hook_invisit that is called after the tour
5    of the left subtree (if any), yet before the tour of the right subtree (if any).
6
7    Note: Right child is always assigned index 1 in path, even if no left sibling.
8    """
9    def _tour(self, p, d, path):
10     results = [None, None]          # will update with results of recursions
11     self._hook_previsit(p, d, path)                    # "pre visit" for p
12     if self._tree.left(p) is not None:                 # consider left child
13       path.append(0)
14       results[0] = self._tour(self._tree.left(p), d+1, path)
15       path.pop( )
16     self._hook_invisit(p, d, path)                     # "in visit" for p
17     if self._tree.right(p) is not None:                # consider right child
18       path.append(1)
19       results[1] = self._tour(self._tree.right(p), d+1, path)
20       path.pop( )
21     answer = self._hook_postvisit(p, d, path, results)       # "post visit" p
22     return answer
23
24   def _hook_invisit(self, p, d, path): pass            # can be overridden
```

Code Fragment 8.33: A BinaryEulerTour base class providing a specialized tour for binary trees. The original EulerTour base class was given in Code Fragment 8.28.

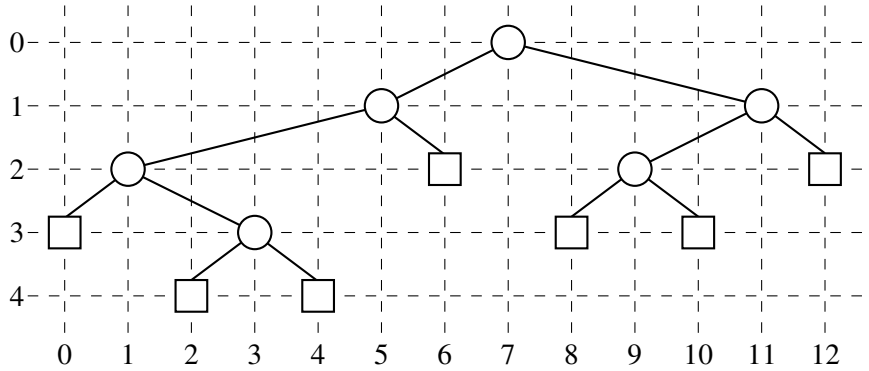


Figure 8.22: An inorder drawing of a binary tree.

To demonstrate use of the BinaryEulerTour framework, we develop a subclass that computes a graphical layout of a binary tree, as shown in Figure 8.22. The geometry is determined by an algorithm that assigns x- and y-coordinates to each position p of a binary tree T using the following two rules:

- $x(p)$ is the number of positions visited before p in an inorder traversal of T.
- $y(p)$ is the depth of p in T.

In this application, we take the convention common in computer graphics that x-coordinates increase left to right and y-coordinates increase top to bottom. So the origin is in the upper left corner of the computer screen.

Code Fragment 8.34 provides an implementation of a BinaryLayout subclass that implements the above algorithm for assigning (x,y) coordinates to the element stored at each position of a binary tree. We adapt the BinaryEulerTour framework by introducing additional state in the form of a _count instance variable that represents the number of "in visits" that we have performed. The x-coordinate for each position is set according to that counter.

```
1  class BinaryLayout(BinaryEulerTour):
2    """Class for computing (x,y) coordinates for each node of a binary tree."""
3    def __init__(self, tree):
4      super().__init__(tree)                # must call the parent constructor
5      self._count = 0                       # initialize count of processed nodes
6
7    def _hook_invisit(self, p, d, path):
8      p.element().setX(self._count)         # x-coordinate serialized by count
9      p.element().setY(d)                   # y-coordinate is depth
10     self._count += 1                      # advance count of processed nodes
```

Code Fragment 8.34: A BinaryLayout class that computes coordinates at which to draw positions of a binary tree. We assume that the element type for the original tree supports setX and setY methods.

Chapter 4

Paths in graphs

4.1 Distances

Depth-first search readily identifies all the vertices of a graph that can be reached from a designated starting point. It also finds explicit paths to these vertices, summarized in its search tree (Figure 4.1). However, these paths might not be the most economical ones possible. In the figure, vertex C is reachable from S by traversing just one edge, while the DFS tree shows a path of length 3. This chapter is about algorithms for finding *shortest paths* in graphs.

Path lengths allow us to talk quantitatively about the extent to which different vertices of a graph are separated from each other:

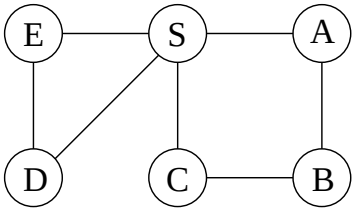
The *distance* between two nodes is the length of the shortest path between them.

To get a concrete feel for this notion, consider a physical realization of a graph that has a ball for each vertex and a piece of string for each edge. If you lift the ball for vertex s high enough, the other balls that get pulled up along with it are precisely the vertices reachable from s. And to find their distances from s, you need only measure how far below s they hang.

In Figure 4.2 for example, vertex B is at distance 2 from S, and there are two shortest paths to it. When S is held up, the strings along each of these paths become taut. On the other hand, edge (D, E) plays no role in any shortest path and therefore remains slack.

Figure 4.1 (a) A simple graph and (b) its depth-first search tree.

(a)



(b)

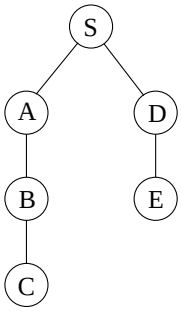


Figure 4.2 A physical model of a graph.

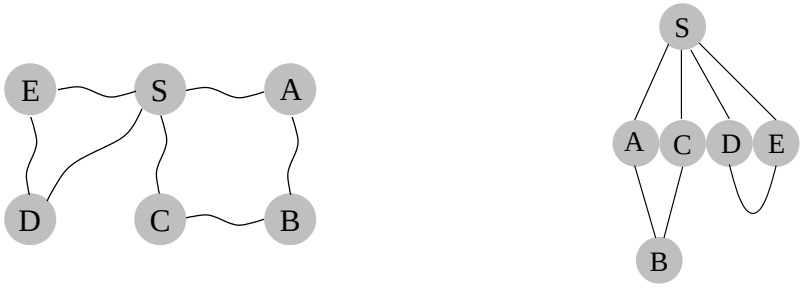


Figure 4.3 Breadth-first search.

```
procedure bfs(G, s)
Input:    Graph G = (V, E), directed or undirected; vertex s ∈ V
Output:   For all vertices u reachable from s, dist(u) is set
          to the distance from s to u.

for all u ∈ V:
   dist(u) = ∞

dist(s) = 0
Q = [s] (queue containing just s)
while Q is not empty:
   u = eject(Q)
   for all edges (u, v) ∈ E:
      if dist(v) = ∞:
         inject(Q, v)
         dist(v) = dist(u) + 1
```

4.2 Breadth-first search

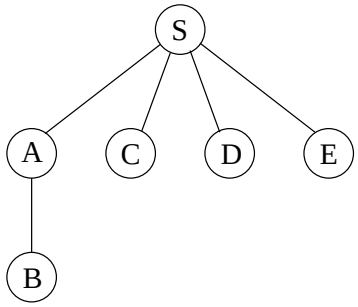
In Figure 4.2, the lifting of s partitions the graph into layers: s itself, the nodes at distance 1 from it, the nodes at distance 2 from it, and so on. A convenient way to compute distances from s to the other vertices is to proceed layer by layer. Once we have picked out the nodes at distance $0, 1, 2, \ldots, d$, the ones at $d + 1$ are easily determined: they are precisely the as-yet-unseen nodes that are adjacent to the layer at distance d. This suggests an iterative algorithm in which two layers are active at any given time: some layer d, which has been fully identified, and $d + 1$, which is being discovered by scanning the neighbors of layer d.

Breadth-first search (BFS) directly implements this simple reasoning (Figure 4.3). Initially the queue Q consists only of s, the one node at distance 0. And for each subsequent distance $d = 1, 2, 3, \ldots$, there is a point in time at which Q contains all the nodes at distance d and nothing else. As these nodes are processed (ejected off the front of the queue), their as-yet-unseen neighbors are injected into the end of the queue.

Let's try out this algorithm on our earlier example (Figure 4.1) to confirm that it does the

Figure 4.4 The result of breadth-first search on the graph of Figure 4.1.

Order of visitation	Queue contents after processing node
	$[S]$
S	$[A\ C\ D\ E]$
A	$[C\ D\ E\ B]$
C	$[D\ E\ B]$
D	$[E\ B]$
E	$[B]$
B	$[\,]$



right thing. If S is the starting point and the nodes are ordered alphabetically, they get visited in the sequence shown in Figure 4.4. The breadth-first search tree, on the right, contains the edges through which each node is initially discovered. Unlike the DFS tree we saw earlier, it has the property that all its paths from S are the shortest possible. It is therefore a *shortest-path tree*.

Correctness and efficiency

We have developed the basic intuition behind breadth-first search. In order to check that the algorithm works correctly, we need to make sure that it faithfully executes this intuition. What we expect, precisely, is that

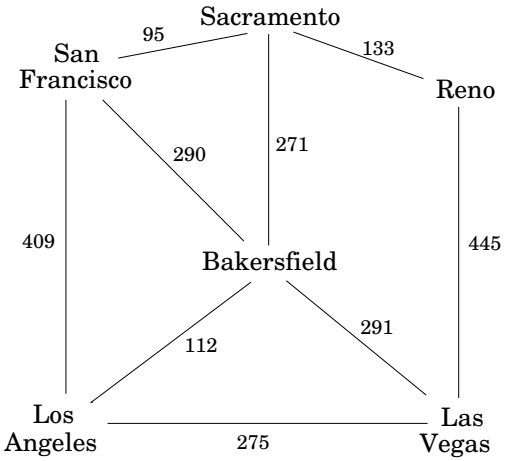
> For each $d = 0, 1, 2, \ldots$, there is a moment at which (1) all nodes at distance $\leq d$ from s have their distances correctly set; (2) all other nodes have their distances set to ∞; and (3) the queue contains exactly the nodes at distance d.

This has been phrased with an inductive argument in mind. We have already discussed both the base case and the inductive step. Can you fill in the details?

The overall running time of this algorithm is linear, $O(|V| + |E|)$, for exactly the same reasons as depth-first search. Each vertex is put on the queue exactly once, when it is first encountered, so there are $2\,|V|$ queue operations. The rest of the work is done in the algorithm's innermost loop. Over the course of execution, this loop looks at each edge once (in directed graphs) or twice (in undirected graphs), and therefore takes $O(|E|)$ time.

Now that we have both BFS and DFS before us: how do their exploration styles compare? Depth-first search makes deep incursions into a graph, retreating only when it runs out of new nodes to visit. This strategy gives it the wonderful, subtle, and extremely useful properties we saw in the Chapter 3. But it also means that DFS can end up taking a long and convoluted route to a vertex that is actually very close by, as in Figure 4.1. Breadth-first search makes sure to visit vertices in increasing order of their distance from the starting point. This is a broader, shallower search, rather like the propagation of a wave upon water. And it is achieved using almost exactly the same code as DFS—but with a queue in place of a stack.

Figure 4.5 Edge lengths often matter.



Also notice one stylistic difference from DFS: since we are only interested in distances from s, we do not restart the search in other connected components. Nodes not reachable from s are simply ignored.

4.3 Lengths on edges

Breadth-first search treats all edges as having the same length. This is rarely true in applications where shortest paths are to be found. For instance, suppose you are driving from San Francisco to Las Vegas, and want to find the quickest route. Figure 4.5 shows the major highways you might conceivably use. Picking the right combination of them is a shortest-path problem in which the length of each edge (each stretch of highway) is important. For the remainder of this chapter, we will deal with this more general scenario, annotating every edge $e \in E$ with a length l_e. If $e = (u, v)$, we will sometimes also write $l(u, v)$ or l_{uv}.

These l_e's do not have to correspond to physical lengths. They could denote time (driving time between cities) or money (cost of taking a bus), or any other quantity that we would like to conserve. In fact, there are cases in which we need to use negative lengths, but we will briefly overlook this particular complication.

4.4 Dijkstra's algorithm

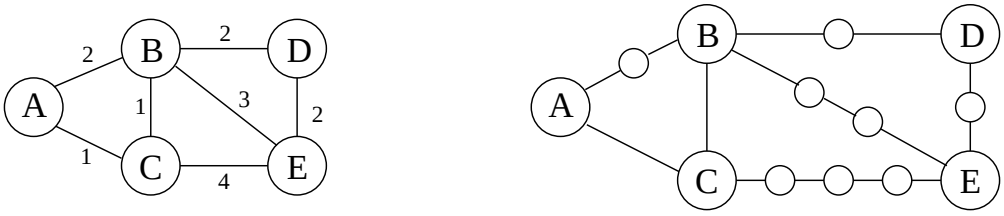
4.4.1 An adaptation of breadth-first search

Breadth-first search finds shortest paths in any graph whose edges have unit length. Can we adapt it to a more general graph $G = (V, E)$ whose edge lengths l_e are *positive integers*?

A more convenient graph

Here is a simple trick for converting G into something BFS can handle: break G's long edges into unit-length pieces, by introducing "dummy" nodes. Figure 4.6 shows an example of this

Figure 4.6 Breaking edges into unit-length pieces.



transformation. To construct the new graph G',

> For any edge $e = (u, v)$ of E, replace it by l_e edges of length 1, by adding $l_e - 1$ dummy nodes between u and v.

Graph G' contains all the vertices V that interest us, and the distances between them are exactly the same as in G. Most importantly, the edges of G' all have unit length. Therefore, we can compute distances in G by running BFS on G'.

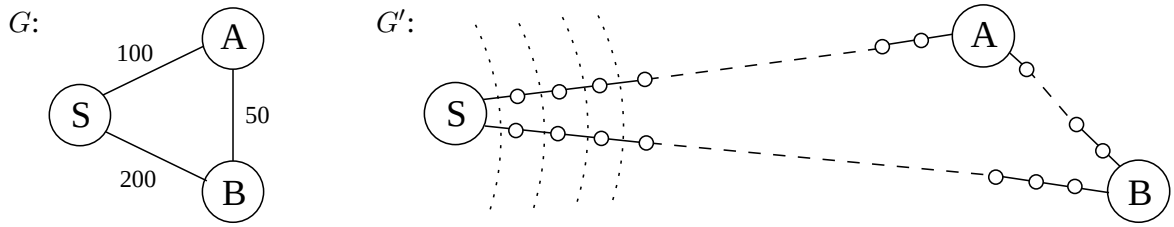
Alarm clocks

If efficiency were not an issue, we could stop here. But when G has very long edges, the G' it engenders is thickly populated with dummy nodes, and the BFS spends most of its time diligently computing distances to these nodes that we don't care about at all.

To see this more concretely, consider the graphs G and G' of Figure 4.7, and imagine that the BFS, started at node s of G', advances by one unit of distance per minute. For the first 99 minutes it tediously progresses along $S - A$ and $S - B$, an endless desert of dummy nodes. Is there some way we can snooze through these boring phases and have an alarm wake us up whenever something *interesting* is happening—specifically, whenever one of the real nodes (from the original graph G) is reached?

We do this by setting two alarms at the outset, one for node A, set to go off at time $T = 100$, and one for B, at time $T = 200$. These are *estimated times of arrival*, based upon the edges currently being traversed. We doze off and awake at $T = 100$ to find A has been discovered. At this point, the estimated time of arrival for B is adjusted to $T = 150$ and we change its alarm accordingly.

Figure 4.7 BFS on G' is mostly uneventful. The dotted lines show some early "wavefronts."



More generally, at any given moment the breadth-first search is advancing along certain edges of G, and there is an alarm for every endpoint node toward which it is moving, set to go off at the estimated time of arrival at that node. Some of these might be overestimates because BFS may later find shortcuts, as a result of future arrivals elsewhere. In the preceding example, a quicker route to B was revealed upon arrival at A. However, *nothing interesting can possibly happen before an alarm goes off*. The sounding of the next alarm must therefore signal the arrival of the wavefront to a real node $u \in V$ by BFS. At that point, BFS might also start advancing along some new edges out of u, and alarms need to be set for their endpoints.

The following "alarm clock algorithm" faithfully simulates the execution of BFS on G'.

- Set an alarm clock for node s at time 0.
- Repeat until there are no more alarms:

  Say the next alarm goes off at time T, for node u. Then:

  - The distance from s to u is T.
  - For each neighbor v of u in G:
    - If there is no alarm yet for v, set one for time $T + l(u, v)$.
    - If v's alarm is set for later than $T + l(u, v)$, then reset it to this earlier time.

Dijkstra's algorithm. The alarm clock algorithm computes distances in any graph with positive integral edge lengths. It is almost ready for use, except that we need to somehow implement the system of alarms. The right data structure for this job is a *priority queue* (usually implemented via a *heap*), which maintains a set of elements (nodes) with associated numeric key values (alarm times) and supports the following operations:

Insert. Add a new element to the set.

Decrease-key. Accommodate the decrease in key value of a particular element.[1]

Delete-min. Return the element with the smallest key, and remove it from the set.

Make-queue. Build a priority queue out of the given elements, with the given key values. (In many implementations, this is significantly faster than inserting the elements one by one.)

The first two let us set alarms, and the third tells us which alarm is next to go off. Putting this all together, we get Dijkstra's algorithm (Figure 4.8).

In the code, `dist`(u) refers to the current alarm clock setting for node u. A value of ∞ means the alarm hasn't so far been set. There is also a special array, `prev`, that holds one crucial piece of information for each node u: the identity of the node immediately before it on the shortest path from s to u. By following these back-pointers, we can easily reconstruct shortest paths, and so this array is a compact summary of all the paths found. A full example of the algorithm's operation, along with the final shortest-path tree, is shown in Figure 4.9.

In summary, we can think of Dijkstra's algorithm as just BFS, except it uses a priority queue instead of a regular queue, so as to prioritize nodes in a way that takes edge lengths

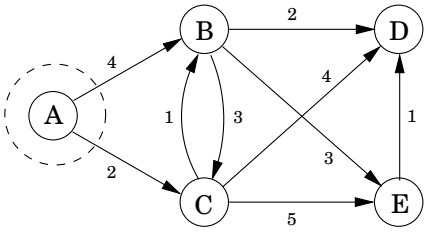
[1]The name *decrease-key* is standard but is a little misleading: the priority queue typically does not itself change key values. What this procedure really does is to notify the queue that a certain key value has been decreased.

into account. This viewpoint gives a concrete appreciation of how and why the algorithm works, but there is a more direct, more abstract derivation that doesn't depend upon BFS at all. We now start from scratch with this complementary interpretation.

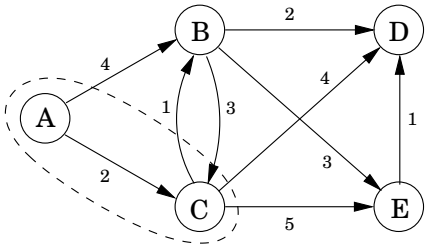
Figure 4.8 Dijkstra's shortest-path algorithm.

```
procedure dijkstra(G, l, s)
Input:    Graph G = (V, E), directed or undirected;
          positive edge lengths {l_e : e ∈ E}; vertex s ∈ V
Output:   For all vertices u reachable from s, dist(u) is set
          to the distance from s to u.

for all u ∈ V:
   dist(u) = ∞
   prev(u) = nil
dist(s) = 0

H = makequeue(V)   (using dist-values as keys)
while H is not empty:
   u = deletemin(H)
   for all edges (u, v) ∈ E:
      if dist(v) > dist(u) + l(u, v):
         dist(v) = dist(u) + l(u, v)
         prev(v) = u
         decreasekey(H, v)
```

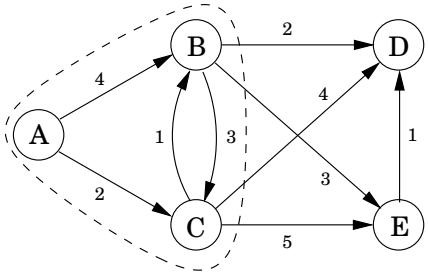
Figure 4.9 A complete run of Dijkstra's algorithm, with node A as the starting point. Also shown are the associated `dist` values and the final shortest-path tree.



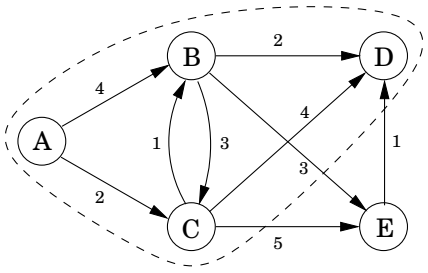
A: 0	D: ∞
B: 4	E: ∞
C: 2	



A: 0	D: 6
B: 3	E: 7
C: 2	



A: 0	D: 5
B: 3	E: 6
C: 2	



A: 0	D: 5
B: 3	E: 6
C: 2	

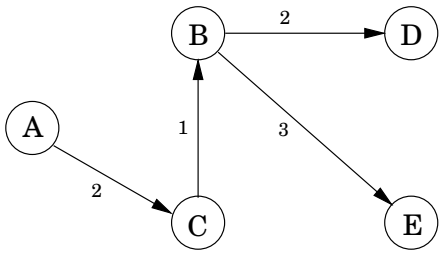
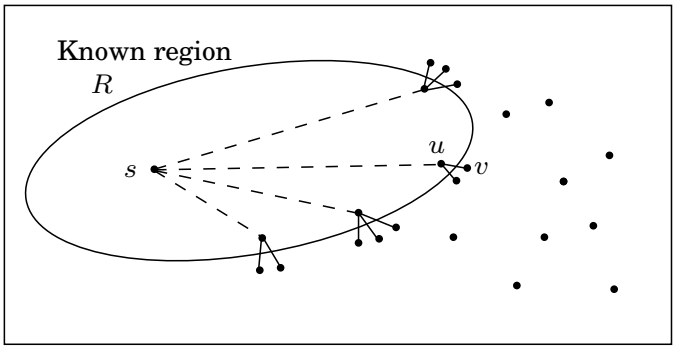


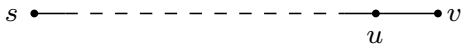
Figure 4.10 Single-edge extensions of known shortest paths.



4.4.2 An alternative derivation

Here's a plan for computing shortest paths: expand outward from the starting point s, steadily growing the region of the graph to which distances and shortest paths are known. This growth should be orderly, first incorporating the closest nodes and then moving on to those further away. More precisely, when the "known region" is some subset of vertices R that includes s, the next addition to it should be *the node outside R that is closest to s*. Let us call this node v; the question is: how do we identify it?

To answer, consider u, the node just before v in the shortest path from s to v:



Since we are assuming that all edge lengths are positive, u must be closer to s than v is. This means that u is in R—otherwise it would contradict v's status as the closest node to s outside R. So, the shortest path from s to v is simply *a known shortest path extended by a single edge.*

But there will typically be many single-edge extensions of the currently known shortest paths (Figure 4.10); which of these identifies v? The answer is, *the shortest of these extended paths*. Because, if an even shorter single-edge-extended path existed, this would once more contradict v's status as the node outside R closest to s. So, it's easy to find v: it is the node outside R for which the smallest value of distance$(s, u) + l(u, v)$ is attained, as u ranges over R. In other words, *try all single-edge extensions of the currently known shortest paths, find the shortest such extended path, and proclaim its endpoint to be the next node of R.*

We now have an algorithm for growing R by looking at extensions of the current set of shortest paths. Some extra efficiency comes from noticing that on any given iteration, the only *new* extensions are those involving the node most recently added to region R. All other extensions will have been assessed previously and do not need to be recomputed. In the following pseudocode, `dist`(v) is the length of the currently shortest single-edge-extended path leading to v; it is ∞ for nodes not adjacent to R.

```
Initialize dist(s) to 0, other dist(·) values to ∞
R = { } (the ''known region'')
while R ≠ V:
   Pick the node v ∉ R with smallest dist(·)
   Add v to R
   for all edges (v, z) ∈ E:
      if dist(z) > dist(v) + l(v, z):
         dist(z) = dist(v) + l(v, z)
```

Incorporating priority queue operations gives us back Dijkstra's algorithm (Figure 4.8).

To justify this algorithm formally, we would use a proof by induction, as with breadth-first search. Here's an appropriate inductive hypothesis.

> At the end of each iteration of the while loop, the following conditions hold: (1) there is a value d such that all nodes in R are at distance $\leq d$ from s and all nodes outside R are at distance $\geq d$ from s, and (2) for every node u, the value `dist`(u) is the length of the shortest path from s to u whose intermediate nodes are constrained to be in R (if no such path exists, the value is ∞).

The base case is straightforward (with $d = 0$), and the details of the inductive step can be filled in from the preceding discussion.

4.4.3 Running time

At the level of abstraction of Figure 4.8, Dijkstra's algorithm is structurally identical to breadth-first search. However, it is slower because the priority queue primitives are computationally more demanding than the constant-time `eject`'s and `inject`'s of BFS. Since `makequeue` takes at most as long as $|V|$ `insert` operations, we get a total of $|V|$ `deletemin` and $|V| + |E|$ `insert`/`decreasekey` operations. The time needed for these varies by implementation; for instance, a binary heap gives an overall running time of $O((|V| + |E|) \log |V|)$.

Which heap is best?

The running time of Dijkstra's algorithm depends heavily on the priority queue implementation used. Here are the typical choices.

Implementation	`deletemin`	`insert`/ `decreasekey`	$\|V\| \times$ `deletemin` + $(\|V\| + \|E\|) \times$ `insert`
Array	$O(\|V\|)$	$O(1)$	$O(\|V\|^2)$
Binary heap	$O(\log \|V\|)$	$O(\log \|V\|)$	$O((\|V\| + \|E\|) \log \|V\|)$
d-ary heap	$O(\frac{d \log \|V\|}{\log d})$	$O(\frac{\log \|V\|}{\log d})$	$O((\|V\| \cdot d + \|E\|) \frac{\log \|V\|}{\log d})$
Fibonacci heap	$O(\log \|V\|)$	$O(1)$ (amortized)	$O(\|V\| \log \|V\| + \|E\|)$

So for instance, even a naive array implementation gives a respectable time complexity of $O(|V|^2)$, whereas with a binary heap we get $O((|V| + |E|) \log |V|)$. Which is preferable?

This depends on whether the graph is *sparse* (has few edges) or *dense* (has lots of them). For all graphs, $|E|$ is less than $|V|^2$. If it is $\Omega(|V|^2)$, then clearly the array implementation is the faster. On the other hand, the binary heap becomes preferable as soon as $|E|$ dips below $|V|^2/\log |V|$.

The d-ary heap is a generalization of the binary heap (which corresponds to $d = 2$) and leads to a running time that is a function of d. The optimal choice is $d \approx |E|/|V|$; in other words, to optimize we must set the degree of the heap to be equal to the *average degree* of the graph. This works well for both sparse and dense graphs. For very sparse graphs, in which $|E| = O(|V|)$, the running time is $O(|V| \log |V|)$, as good as with a binary heap. For dense graphs, $|E| = \Omega(|V|^2)$ and the running time is $O(|V|^2)$, as good as with a linked list. Finally, for graphs with intermediate density $|E| = |V|^{1+\delta}$, the running time is $O(|E|)$, linear!

The last line in the table gives running times using a sophisticated data structure called a *Fibonacci heap*. Although its efficiency is impressive, this data structure requires considerably more work to implement than the others, and this tends to dampen its appeal in practice. We will say little about it except to mention a curious feature of its time bounds. Its `insert` operations take varying amounts of time but are guaranteed to *average* $O(1)$ over the course of the algorithm. In such situations (one of which we shall encounter in Chapter 5) we say that the *amortized* cost of heap `insert`'s is $O(1)$.

4.5 Priority queue implementations

4.5.1 Array

The simplest implementation of a priority queue is as an unordered array of key values for all potential elements (the vertices of the graph, in the case of Dijkstra's algorithm). Initially, these values are set to ∞.

An `insert` or `decreasekey` is fast, because it just involves adjusting a key value, an $O(1)$ operation. To `deletemin`, on the other hand, requires a linear-time scan of the list.

4.5.2 Binary heap

Here elements are stored in a *complete* binary tree, namely, a binary tree in which each level is filled in from left to right, and must be full before the next level is started. In addition, a special ordering constraint is enforced: *the key value of any node of the tree is less than or equal to that of its children*. In particular, therefore, the root always contains the smallest element. See Figure 4.11(a) for an example.

To `insert`, place the new element at the bottom of the tree (in the first available position), and let it "bubble up." That is, if it is smaller than its parent, swap the two and repeat (Figure 4.11(b)–(d)). The number of swaps is at most the height of the tree, which is $\lfloor \log_2 n \rfloor$ when there are n elements. A `decreasekey` is similar, except that the element is already in the tree, so we let it bubble up from its current position.

To `deletemin`, return the root value. To then remove this element from the heap, take the last node in the tree (in the rightmost position in the bottom row) and place it at the root. Let it "sift down": if it is bigger than either child, swap it with the smaller child and repeat (Figure 4.11(e)–(g)). Again this takes $O(\log n)$ time.

The regularity of a complete binary tree makes it easy to represent using an array. The tree nodes have a natural ordering: row by row, starting at the root and moving left to right within each row. If there are n nodes, this ordering specifies their positions $1, 2, \ldots, n$ within the array. Moving up and down the tree is easily simulated on the array, using the fact that node number j has parent $\lfloor j/2 \rfloor$ and children $2j$ and $2j + 1$ (Exercise 4.16).

4.5.3 d-ary heap

A d-ary heap is identical to a binary heap, except that nodes have d children instead of just two. This reduces the height of a tree with n elements to $\Theta(\log_d n) = \Theta((\log n)/(\log d))$. Inserts are therefore speeded up by a factor of $\Theta(\log d)$. Deletemin operations, however, take a little longer, namely $O(d \log_d n)$ (do you see why?).

The array representation of a binary heap is easily extended to the d-ary case. This time, node number j has parent $\lceil (j-1)/d \rceil$ and children $\{(j-1)d + 2, \ldots, \min\{n, (j-1)d + d + 1\}\}$ (Exercise 4.16).

Figure 4.11 (a) A binary heap with 10 elements. Only the key values are shown. (b)–(d) The intermediate "bubble-up" steps in inserting an element with key 7. (e)–(g) The "sift-down" steps in a delete-min operation.

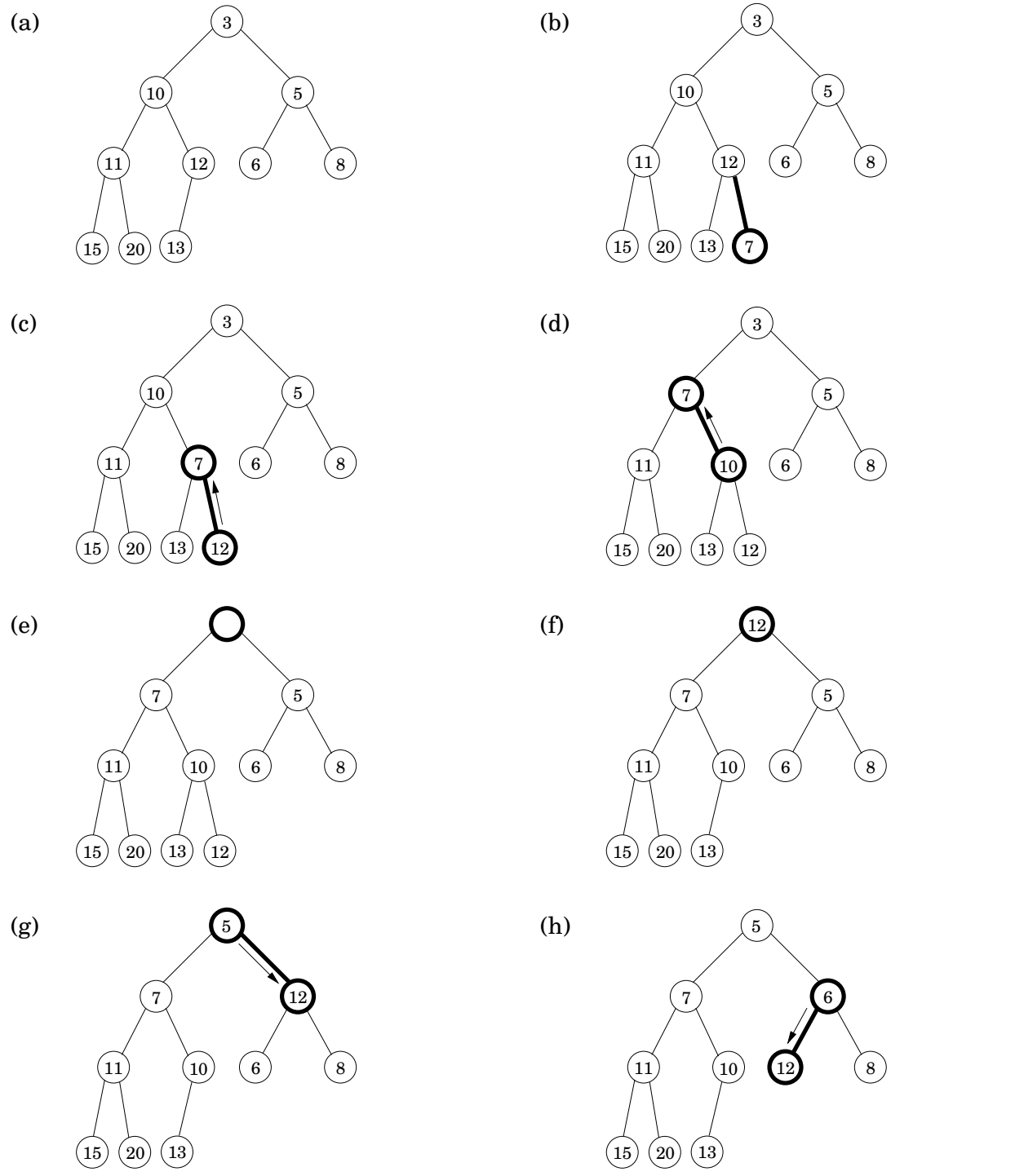
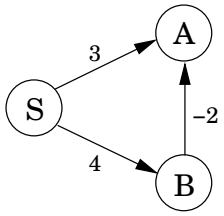


Figure 4.12 Dijkstra's algorithm will not work if there are negative edges.



4.6 Shortest paths in the presence of negative edges

4.6.1 Negative edges

Dijkstra's algorithm works in part because the shortest path from the starting point s to any node v must pass exclusively through nodes that are closer than v. This no longer holds when edge lengths can be negative. In Figure 4.12, the shortest path from S to A passes through B, a node that is further away!

What needs to be changed in order to accommodate this new complication? To answer this, let's take a particular high-level view of Dijkstra's algorithm. A crucial invariant is that the `dist` values it maintains are always either overestimates or exactly correct. They start off at ∞, and the only way they ever change is by updating along an edge:

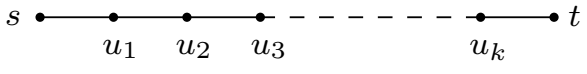
`procedure update(`$(u, v) \in E$`)`
`dist(`v`)` $= \min\{$`dist(`v`)`$,$ `dist(`u`)` $+ l(u, v)\}$

This *update* operation is simply an expression of the fact that the distance to v cannot possibly be more than the distance to u, plus $l(u, v)$. It has the following properties.

1. It gives the correct distance to v in the particular case where u is the second-last node in the shortest path to v, and `dist`(u) is correctly set.

2. It will never make `dist`(v) too small, and in this sense it is *safe*. For instance, a slew of extraneous `update`'s can't hurt.

This operation is extremely useful: it is harmless, and if used carefully, will correctly set distances. In fact, Dijkstra's algorithm can be thought of simply as a sequence of `update`'s. We know this particular sequence doesn't work with negative edges, but is there some other sequence that does? To get a sense of the properties this sequence must possess, let's pick a node t and look at the shortest path to it from s.



This path can have at most $|V| - 1$ edges (do you see why?). If the sequence of updates performed includes $(s, u_1), (u_1, u_2), (u_2, u_3), \ldots, (u_k, t)$, *in that order* (though not necessarily consecutively), then by the first property the distance to t will be correctly computed. It doesn't

Figure 4.13 The Bellman-Ford algorithm for single-source shortest paths in general graphs.

```
procedure shortest-paths(G, l, s)
Input:    Directed graph G = (V, E);
          edge lengths {l_e : e ∈ E} with no negative cycles;
          vertex s ∈ V
Output:   For all vertices u reachable from s, dist(u) is set
          to the distance from s to u.

for all u ∈ V:
   dist(u) = ∞
   prev(u) = nil

dist(s) = 0
repeat |V| − 1 times:
   for all e ∈ E:
      update(e)
```

matter what other updates occur on these edges, or what happens in the rest of the graph, because updates are *safe*.

But still, if we don't know all the shortest paths beforehand, how can we be sure to update the right edges in the right order? Here is an easy solution: simply update *all* the edges, $|V| - 1$ times! The resulting $O(|V| \cdot |E|)$ procedure is called the Bellman-Ford algorithm and is shown in Figure 4.13, with an example run in Figure 4.14.

A note about implementation: for many graphs, the maximum number of edges in any shortest path is substantially less than $|V| - 1$, with the result that fewer rounds of updates are needed. Therefore, it makes sense to add an extra check to the shortest-path algorithm, to make it terminate immediately after any round in which no update occurred.

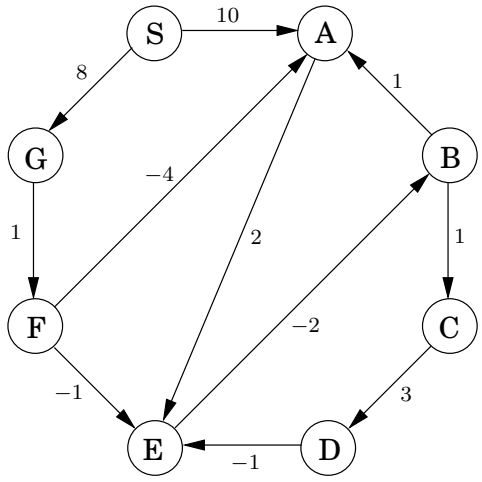
4.6.2 Negative cycles

If the length of edge (E, B) in Figure 4.14 were changed to -4, the graph would have a *negative cycle* $A \to E \to B \to A$. In such situations, it doesn't make sense to even ask about shortest paths. There is a path of length 2 from A to E. But going round the cycle, there's also a path of length 1, and going round multiple times, we find paths of lengths $0, -1, -2$, and so on.

The shortest-path problem is ill-posed in graphs with negative cycles. As might be expected, our algorithm from Section 4.6.1 works only in the absence of such cycles. But where did this assumption appear in the derivation of the algorithm? Well, it slipped in when we asserted the *existence* of a shortest path from s to t.

Fortunately, it is easy to automatically detect negative cycles and issue a warning. Such a cycle would allow us to endlessly apply rounds of `update` operations, reducing `dist` estimates every time. So instead of stopping after $|V| - 1$ iterations, perform one extra round. There is a negative cycle if and only if some `dist` value is reduced during this final round.

Figure 4.14 The Bellman-Ford algorithm illustrated on a sample graph.



	Iteration							
Node	0	1	2	3	4	5	6	7
S	0	0	0	0	0	0	0	0
A	∞	10	10	5	5	5	5	5
B	∞	∞	∞	10	6	5	5	5
C	∞	∞	∞	∞	11	7	6	6
D	∞	∞	∞	∞	∞	14	10	9
E	∞	∞	12	8	7	7	7	7
F	∞	∞	9	9	9	9	9	9
G	∞	8	8	8	8	8	8	8

4.7 Shortest paths in dags

There are two subclasses of graphs that automatically exclude the possibility of negative cycles: graphs without negative edges, and graphs without cycles. We already know how to efficiently handle the former. We will now see how the single-source shortest-path problem can be solved in just linear time on directed acyclic graphs.

As before, we need to perform a sequence of updates that includes every shortest path as a subsequence. The key source of efficiency is that

> In any path of a dag, the vertices appear in increasing linearized order.

Therefore, it is enough to linearize (that is, topologically sort) the dag by depth-first search, and then visit the vertices in sorted order, updating the edges out of each. The algorithm is given in Figure 4.15.

Notice that our scheme doesn't require edges to be positive. In particular, we can find *longest paths* in a dag by the same algorithm: just negate all edge lengths.

Figure 4.15 A single-source shortest-path algorithm for directed acyclic graphs.

```
procedure dag-shortest-paths(G, l, s)
Input:    Dag G = (V, E);
          edge lengths {l_e : e ∈ E}; vertex s ∈ V
Output:   For all vertices u reachable from s, dist(u) is set
          to the distance from s to u.

for all u ∈ V:
   dist(u) = ∞
   prev(u) = nil

dist(s) = 0
Linearize G
for each u ∈ V, in linearized order:
   for all edges (u, v) ∈ E:
      update(u, v)
```

Python heapq HOWTO — Excerpt

Python's heapq module provides an implementation of the **heap queue algorithm**, also known as a **priority queue**. A heap is a binary tree with the property that each parent node is less than or equal to its children (for a *min-heap*). The smallest element is always at the root.

Unlike specialized data structures in other languages, Python's heapq works on regular lists. It maintains the **heap invariant** through simple functions that modify the list in-place.

Key Functions

- heapq.heapify(list)
    - Transforms a list into a valid min-heap in-place.
    - Example:
    - import heapq
    - data = [5, 3, 8, 1, 2]
    - heapq.heapify(data)
    - print(data) # [1, 2, 8, 5, 3] — heap order
- heapq.heappush(heap, item)
    - Adds an item while maintaining the heap invariant.
    - Example:
    - heapq.heappush(data, 0)
    - print(data) # [0, 1, 8, 5, 3, 2]
- heapq.heappop(heap)
    - Removes and returns the smallest item.
    - Example:
    - smallest = heapq.heappop(data)
    - print(smallest) # 0
- heapq.heappushpop(heap, item)
    - Pushes new item on the heap and pops the smallest item in one atomic operation. More efficient than push followed by pop.
- heapq.heapreplace(heap, item)
    - Pops and returns the smallest item, then pushes the new item. Slightly different from heappushpop.
- heapq.nlargest(n, iterable[, key])
    - Returns the n largest elements from the dataset.
    - Example:

    - heapq.nlargest(3, [1, 5, 2, 8, 3]) # [8, 5, 3]
- heapq.nsmallest(n, iterable[, key])
    - Returns the n smallest elements.

Use Cases

- Priority queues (e.g., task scheduling)
- Finding top-N or bottom-N elements in large datasets
- Graph algorithms like Dijkstra's shortest path

Notes

- By default, heapq creates a **min-heap**. For a max-heap, you can invert the values:
- max_heap = []
- for item in data:
- heapq.heappush(max_heap, -item)
- largest = -heapq.heappop(max_heap)

The heapq module is part of Python's standard library and is highly efficient for in-place, memory-friendly priority queue operations.

Case Study: Heap-Based Job Scheduler

Overview

A **job scheduler** manages a queue of tasks (jobs) and decides which job should be processed next. One of the most efficient ways to manage jobs with priorities is using a **heap**—a special tree-based data structure.

In this case study, we'll explore:

- What a **heap-based job scheduler** does
- How its **UML class diagram** is structured
- What **test cases** are written to check if it works properly

1. System Description in Simple Terms

A **Job Scheduler** has the following features:

- Stores jobs with different priorities.
- Always runs the **highest priority** job first.
- Uses a **Min Heap** or **Max Heap** (we'll use Max Heap here).
- Supports adding, removing, and checking jobs.

Example Jobs:

Job ID	Job Name	Priority
1	Email Sync	2
2	Backup Files	5
3	Virus Scan	3

In a **Max Heap**, the **Backup Files** job will be scheduled first.

2. UML Diagram Breakdown

Here's a simplified description of the UML classes (in plain English):

Job Class:

- Represents a single job/task.
- **Attributes:**
    - id (number)
    - name (text)
    - priority (number)

- **Methods:**
    - Getters and setters

JobScheduler Class:

- Manages all the jobs using a heap.
- **Attributes:**
    - jobHeap → A list or array that keeps the heap.
- **Methods:**
    - addJob(Job job) → Adds a job to the heap.
    - getNextJob() → Returns and removes the highest-priority job.
    - peekNextJob() → Shows the highest-priority job without removing.
    - isEmpty() → Checks if there are jobs left.

3. How UML Maps to Test Cases

Now let's see how each part of the UML is tested.

Test Case 1: Add a Job

- **UML Method:** addJob(Job job)
- **Test:** Add a job and check if it's in the heap.
- **Example:** Add "Email Sync" with priority 2 → Heap should contain 1 job.

Test Case 2: Get the Next Job

- **UML Method:** getNextJob()
- **Test:** Add multiple jobs, get the one with highest priority.
- **Example:** Add 3 jobs → call getNextJob() → should return the one with priority 5.

Test Case 3: Peek Without Removing

- **UML Method:** peekNextJob()
- **Test:** Should show the highest-priority job but not remove it.
- **Example:** Call peekNextJob() twice → should give the same job both times.

Test Case 4: Empty Scheduler

- **UML Method:** isEmpty()
- **Test:** Should return true if no jobs are present.
- **Example:** After removing all jobs → isEmpty() should return true.